



HTTP

(Hyper Text Transfer Protocol)



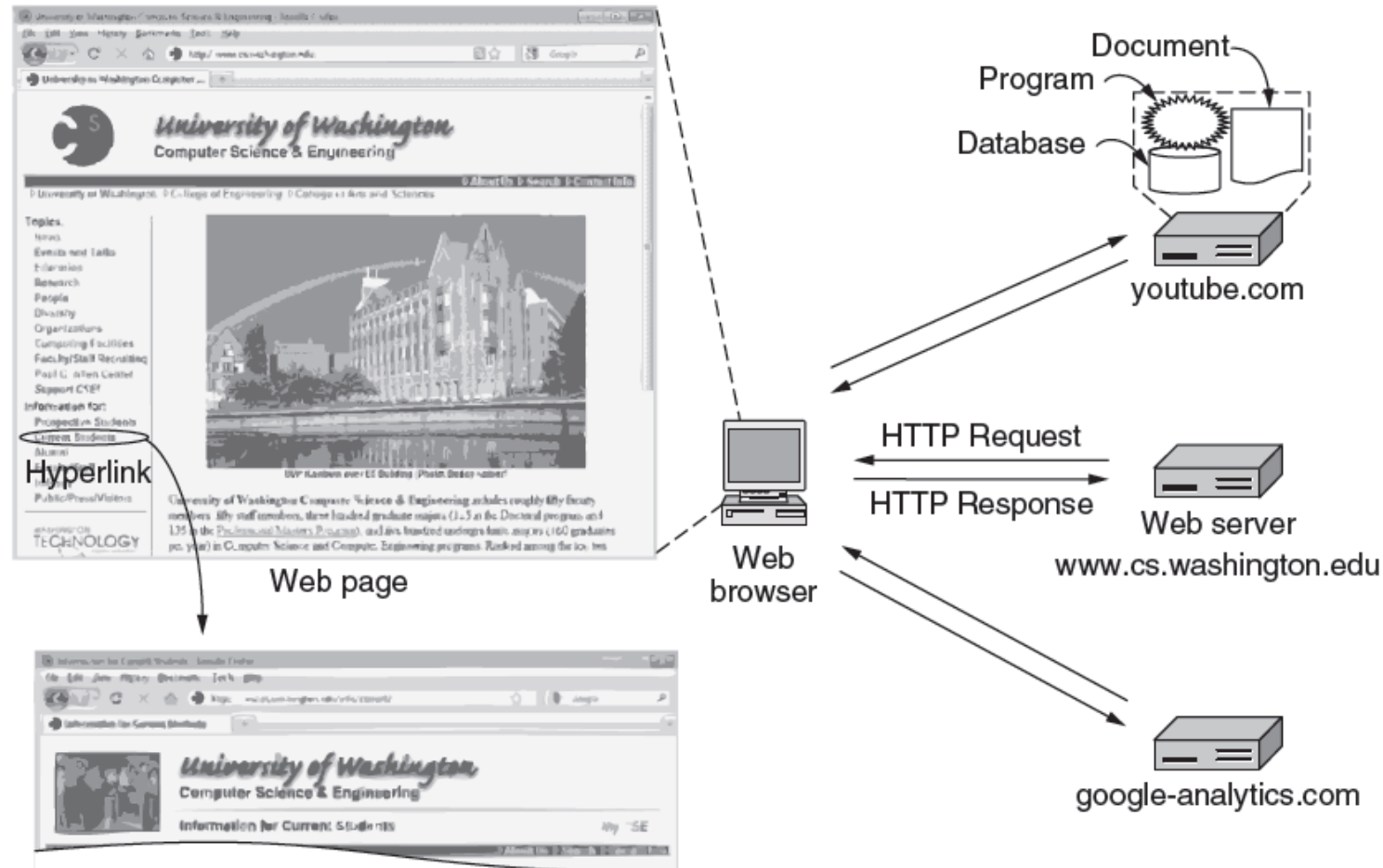
Objectives & Goals

- Understand how the Web evolved
- Discuss and describe the HTTP protocol and each of its versions
- Distinguish between URI/URL/URN
- Distinguish between HTML and HTTP
- Be able to analyse the HTTP communication
- Describe different HTTP methods and error codes
- Discuss how the Transport Layer impacts the HTTP performance
- Discuss a new generation of protocols for content delivery (including Transport protocols)
- Capture packets/streams and discuss the outcome of them



WWW Architectural Overview

- HTTP transfers pages from servers to browsers





Evolution of the Web

- Earliest websites provided static content with little additional media

World Wide Web

The WorldWideWeb (W3) is a wide-area [hypermedia](#) information retrieval initiative aiming to give universal access to a large universe of documents.

Everything there is online about W3 is linked directly or indirectly to this document, including an [executive summary](#) of the project, [Mailing lists](#) , [Policy](#) , November's [W3 news](#) , [Frequently Asked Questions](#) .

[What's out there?](#)

Pointers to the world's online information, [subjects](#) , [W3 servers](#), etc.

[Help](#)

on the browser you are using

[Software Products](#)

A list of W3 project components and their current state. (e.g. [Line Mode](#) ,X11 [Viola](#) , [NeXTStep](#) , [Servers](#) , [Tools](#) , [Mail robot](#) , [Library](#) .)

[Technical](#)

Details of protocols, formats, program internals etc

[Bibliography](#)

Paper documentation on W3 and references.

[People](#)

A list of some people involved in the project.

[History](#)

A summary of the history of the project.

[How can I help ?](#)

If you would like to support the web..

[Getting code](#)

Getting the code by [anonymous FTP](#) , etc.

<http://info.cern.ch/hypertext/WWW/TheProject.html>



Evolution of the Web

What is this type of document?
How is it called?

World Wide Web

The WorldWideWeb (W3) is a wide-area [hypermedia](#) information retrieval initiative aiming to give universal access to a large universe of documents.

Everything there is online about W3 is linked directly or indirectly to this document, including an [executive summary](#) of the project, [Mailing lists](#) , [Policy](#) , November's [W3 news](#) , [Frequently Asked Questions](#) .

[What's out there?](#)

Pointers to the world's online information, [subjects](#) , [W3 servers](#), etc.

[Help](#)

on the browser you are using

[Software Products](#)

A list of W3 project components and their current state. (e.g. [Line Mode](#) ,X11 [Viola](#) , [NeXTStep](#) , [Servers](#) , [Tools](#) , [Mail robot](#) , [Library](#) .)

[Technical](#)

Details of protocols, formats, program internals etc

[Bibliography](#)

Paper documentation on W3 and references.

[People](#)

A list of some people involved in the project.

[History](#)

A summary of the history of the project.

[How can I help ?](#)

If you would like to support the web..

[Getting code](#)

Getting the code by [anonymous FTP](#) , etc.

<http://info.cern.ch/hypertext/WWW/TheProject.html>



HTML (Hyper Text Markup Language)

- It is the standard markup language for documents designed to be displayed in a web browser
- HTML describes the structure of a web page semantically and originally included cues for the appearance of the document.
- A language to create Documents that browsers parse and render

Version	Year
HTML	1991
HTML+	1993
HTML 2.0	1995
HTML 3.2	1997
HTML 4.01	1999
XHTML 1.0	2000
HTML5	2012
XHTML5	2013

(developed in 1980 by Tim Berners-Lee; there are 5 versions)



URN vs URI vs URL

- URN: Uniform Resource Name
 - Globally unique, persistent identifier
 - Independent of location (i.e. independent of the protocol or the server)
- URL: Uniform Resource Locator
 - Refers to an existing protocol (http:, wais:, ftp:, mailto:, gopher:, news:)
 - Points to a document on a specific server and specific location of the **Resource**
- URI: Uniform Resource Identifier
 - Collection of URL's and URN's



URL

<scheme> : //<host> :<port> /<path> ;<parameters> ?<query> #<fragment>

- Scheme - The protocol you are using
- Host - Host name or IP address
- Port - port number that protocol server is using
- Path - Path and filename reference of object on server
- Parameters - Any specific parameters that object needs
- Query - Query string
- Fragment - Reference to a subset (i.e. an anchor) of an object within the document



URL

<scheme> : //<host> :<port> /<path> ;<parameters> ?<query> #<fragment>



Name	Used for	Example
http	Hypertext (HTML)	http://www.ee.uwa.edu/~rob/
https	Hypertext with security	https://www.bank.com/accounts/
ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	Local file	file:///usr/suzanne/prog.c
mailto	Sending email	mailto:JohnUser@acm.org
rtsp	Streaming media	rtsp://youtube.com/montypython.mpg
sip	Multimedia calls	sip:eve@adversary.com
about	Browser information	about:plugins



Evolution of the Web

What is this type of document?
How is it called?

World Wide Web

The WorldWideWeb (W3) is a wide-area [hypermedia](#) information retrieval initiative aiming to give universal access to a large universe of documents.

Everything there is online about W3 is linked directly or indirectly to this document, including an [executive summary](#) of the project, [Mailing lists](#) , [Policy](#) , November's [W3 news](#) , [Frequently Asked Questions](#) .

[What's out there?](#)

Pointers to the world's online information, [subjects](#) , [W3 servers](#), etc.

[Help](#)

on the browser you are using

[Software Products](#)

A list of W3 project components and their current state. (e.g. [Line Mode](#) ,X11 [Viola](#) , [NeXTStep](#) , [Servers](#) , [Tools](#) , [Mail robot](#) , [Library](#) .)

[Technical](#)

Details of protocols, formats, program internals etc

[Bibliography](#)

Paper documentation on W3 and references.

[People](#)

A list of some people involved in the project.

[History](#)

A summary of the history of the project.

[How can I help ?](#)

If you would like to support the web..

[Getting code](#)

Getting the code by [anonymous FTP](#) , etc.

<http://info.cern.ch/hypertext/WWW/TheProject.html>



Evolution of the Web

- Earliest websites provided static content with little additional media
- Over time, websites grew to include many more things, like deepening the web structure (adding more pages), adding images, logos, and even started serving some dynamic content



ESPN in 1996



Evolution of the Web

The image shows a screenshot of the Público website, a Portuguese news outlet. The website features a header with the Público logo, navigation links (AZUL, P2, ÍPSILON, ÍMPAR, FUGAS, P3, PODCASTS), and buttons for 'Assinar' (Subscribe) and 'Entrar' (Login). Below the header is a main navigation bar with categories like POLÍTICA, SOCIEDADE, LOCAL, ECONOMIA, MUNDO, CULTURA, DESPORTO, and MAIS. The main content area displays a large banner for '#DiadaJuventude' (Youth Day) with a '15% de desconto' (15% discount) offer, valid until 15/08. The banner includes the flytap.com logo. Below the banner is a section titled 'GUERRA NA UCRÂNIA' (War in Ukraine) with a sub-header 'AO MINUTO' (Minute by minute) and a brief description: 'União Europeia e mais 42 países exigem "retirada imediata" das t.' (European Union and more 42 countries demand "immediate withdrawal" of the t.). To the right of the website, a browser developer tool is open, showing the HTML and CSS of the page. The HTML panel displays the document structure, including the <html> tag with various attributes, the <body> tag, and the <div> tags for the header, main content, and footer. The CSS panel shows the styles applied to the selected element, including display, flex, justify-content, align-items, padding, margin, width, height, and font-family. The CSS panel also includes a 'Filter' section with a search bar and a list of styles.

Modern websites are incredibly complex and rely on often hundreds of resources to properly function



Evolution of the Web

Static Web Pages

Item	HTML 1.0	HTML 2.0	HTML 3.0	HTML 4.0	HTML 5.0
Hyperlinks	x	x	x	x	x
Images	x	x	x	x	x
Lists	x	x	x	x	x
Active maps & images		x	x	x	x
Forms		x	x	x	x
Equations			x	x	x
Toolbars			x	x	x
Tables			x	x	x
Accessibility features				x	x
Object embedding				x	x
Style sheets				x	x
Scripting				x	x
Video and audio					x
Inline vector graphics					x
XML representation					x
Background threads					x
Browser storage					x
Drawing canvas					x

Static Web pages are simply files

- Have the same contents for each viewing

Can be visually rich and interactive nonetheless:

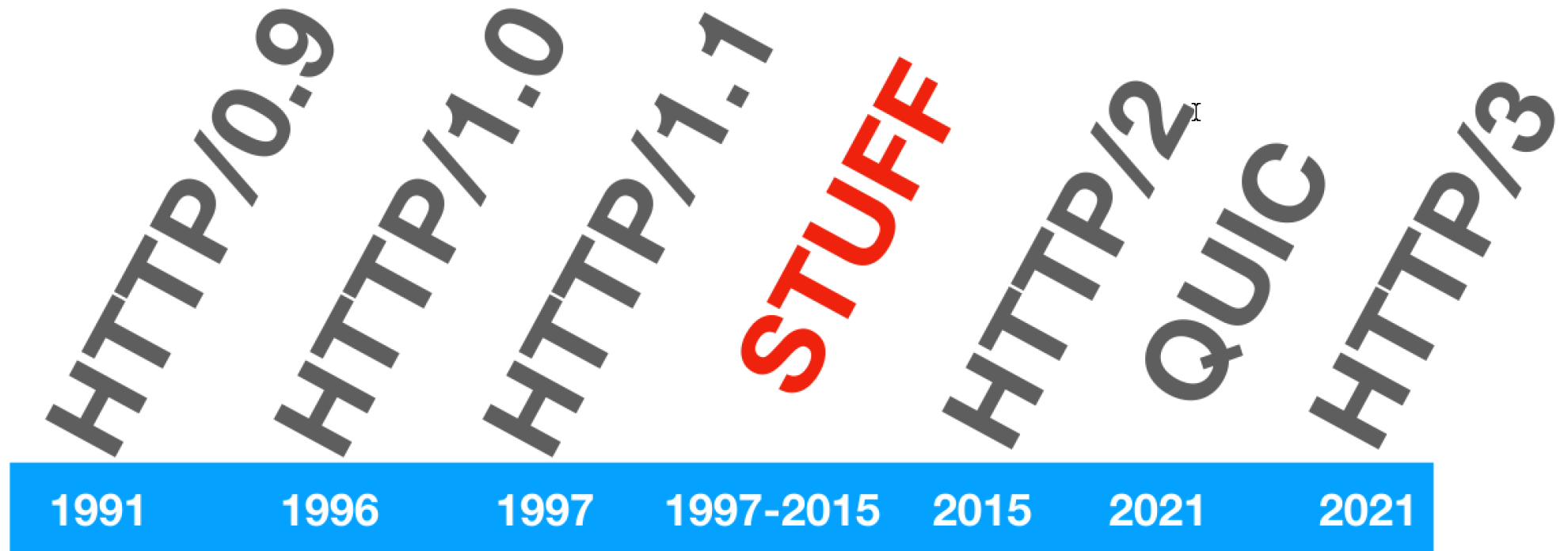
- HTML that mixes text and images - Forms that gather user input
- Style sheets that tailor presentation & Vector graphics, videos . . .

Progression of features through HTML 5.0



Protocol Evolution

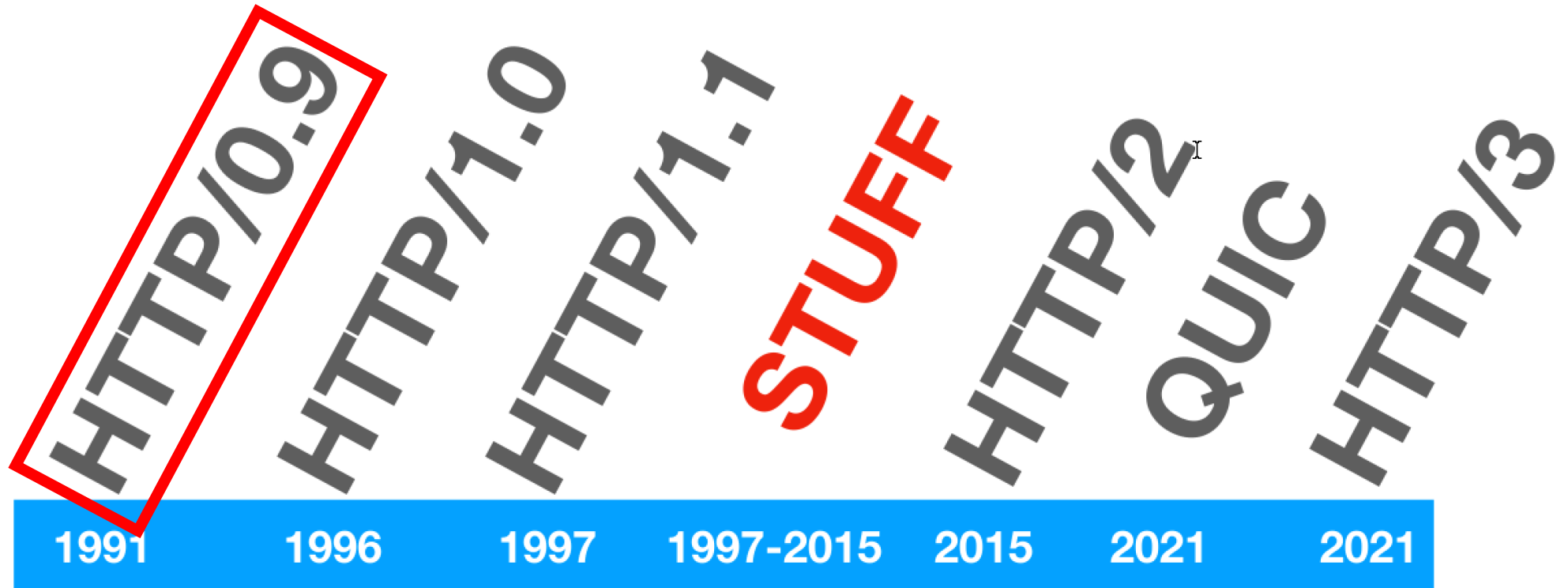
The Web evolved, so protocols needed to evolve as well...





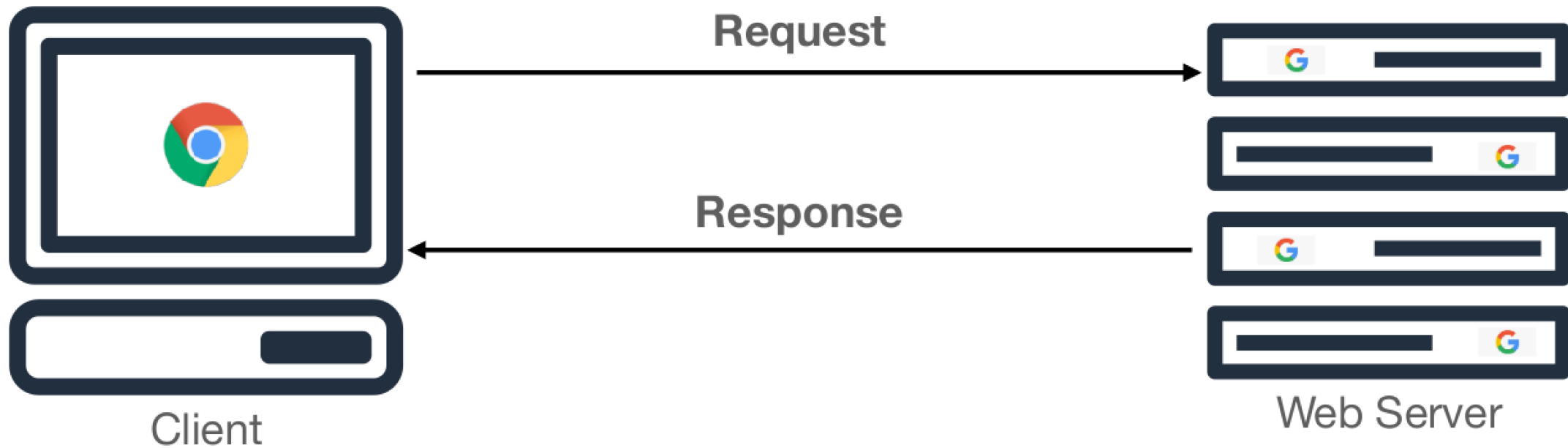
Protocol Evolution

The Web evolved, so protocols needed to evolve as well...



Interfacing with the Web

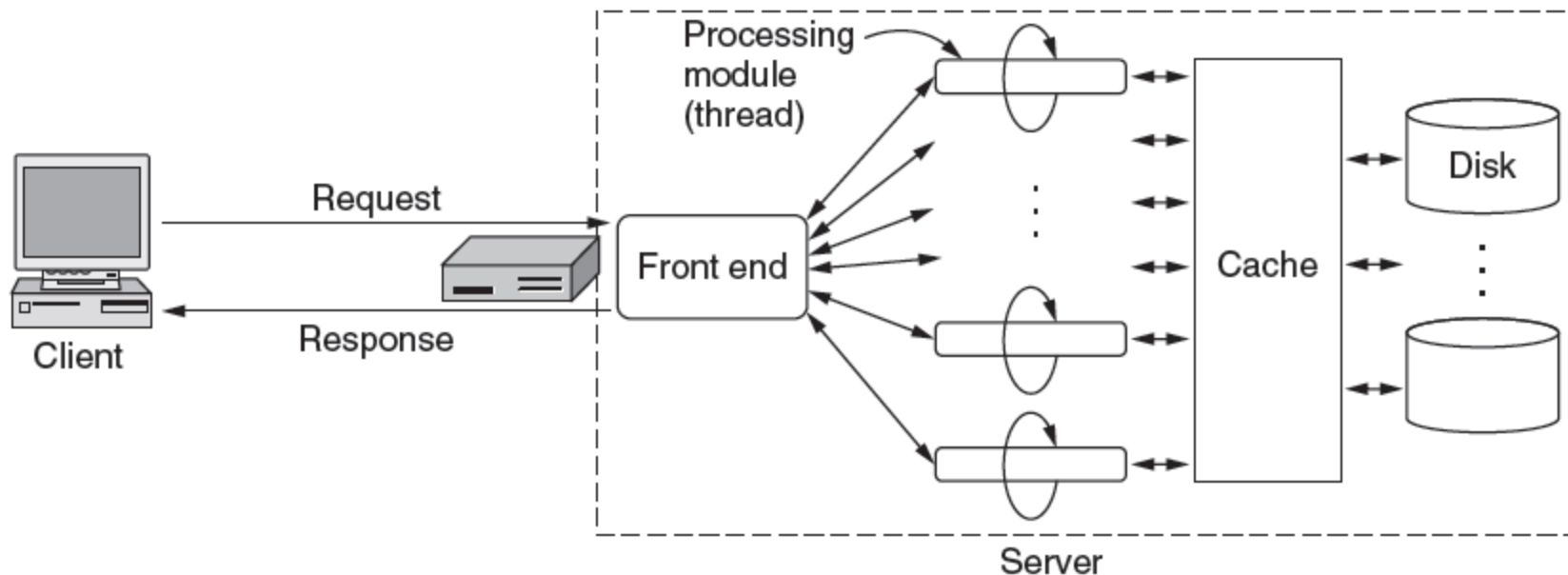
Client / Server Model



Architectural Overview (5)

To scale performance, Web servers can use:

- Caching, multiple threads, and a front end





HTTP/0.9

Single Line Protocol

- In 1991, Tim-Berners Lee needed a simple protocol to test his new invention (the web; remember HTML?)
- Request was a single line command, supported only retrieving HTML content
 - GET /index.html
- Response was the file data itself!
- HTTP/0.9 was built on top of TCP, for reliable transport of data, and the connection was closed after every single request



HTTP/0.9

Single Line Protocol

1. Let's try it?

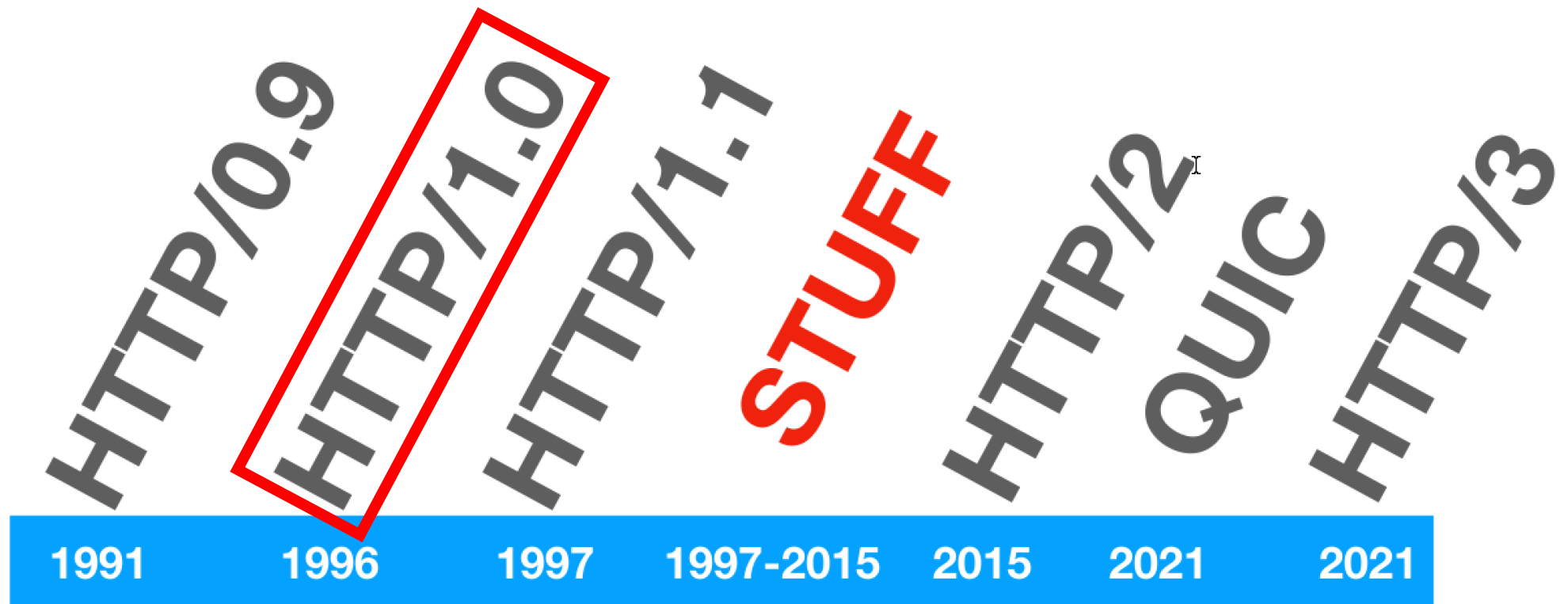
Telnet to Google (remember that it is an ASCII and single line protocol?) and ask him to send you the document which name is ***Doodles...***

2. Draw the **Transport** and **Application** flows that occurs when you ask Google to send you the document identified by *Doodles*



Protocol Evolution

The Web evolved, so protocols needed to evolve as well...





HTTP/1.0 – (1995)

A lot of specification improvements

- Goals: “generic, stateless, object-oriented protocol which can be used for many tasks, such as name servers and distributed object management systems” (from RFC1945)
- Added versioning, a number of new methods (POST, HEAD, PUT, DELETE, LINK, UNLINK), supported myriad different content-types (no longer just HTML!), and included headers to accompany each request and response

Headers

```
Host: localhost:8000
User-Agent: Mozilla/5.0 (Macintosh;... )... Firefox/51.0
Accept: text/html,application/xhtml+xml,...,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Content-Type: multipart/form-data; boundary=-12656974
Content-Length: 345
```

Request headers

General headers

Representation
headers

```
-12656974
(more data)
```



HTTP/1.0 – (1995)

A lot of specification improvements

- Media types
- Status codes
- Caching
- Authorization



HTTP/1.0

Supported myriad different content-types

- MIME (Multipurpose Internet Mail Extension) types which browser can accept
 - text/plain
 - text/html
 - application/postscript
 - image/gif
 - image/jpeg
 - audio/basic
 - video/mpeg
 - x-world/x-vrml



HTTP/1.0

New methods

- GET - Returns object
- POST – Send information to be stored on server or as input to a script
- HEAD – Returns information about object
- PUT – Sends new copy of existing object to server (like an *Update* to the object)
- DELETE – Deletes an object

What do you know about RESTfull APIs?

HTTP/1.0

Response Codes

- Status-codes 2xx – Success
- Status-codes 3xx – Redirection
- Status-codes 4xx - Client error
- Status-codes 5xx - Server error





HTTP/1.0

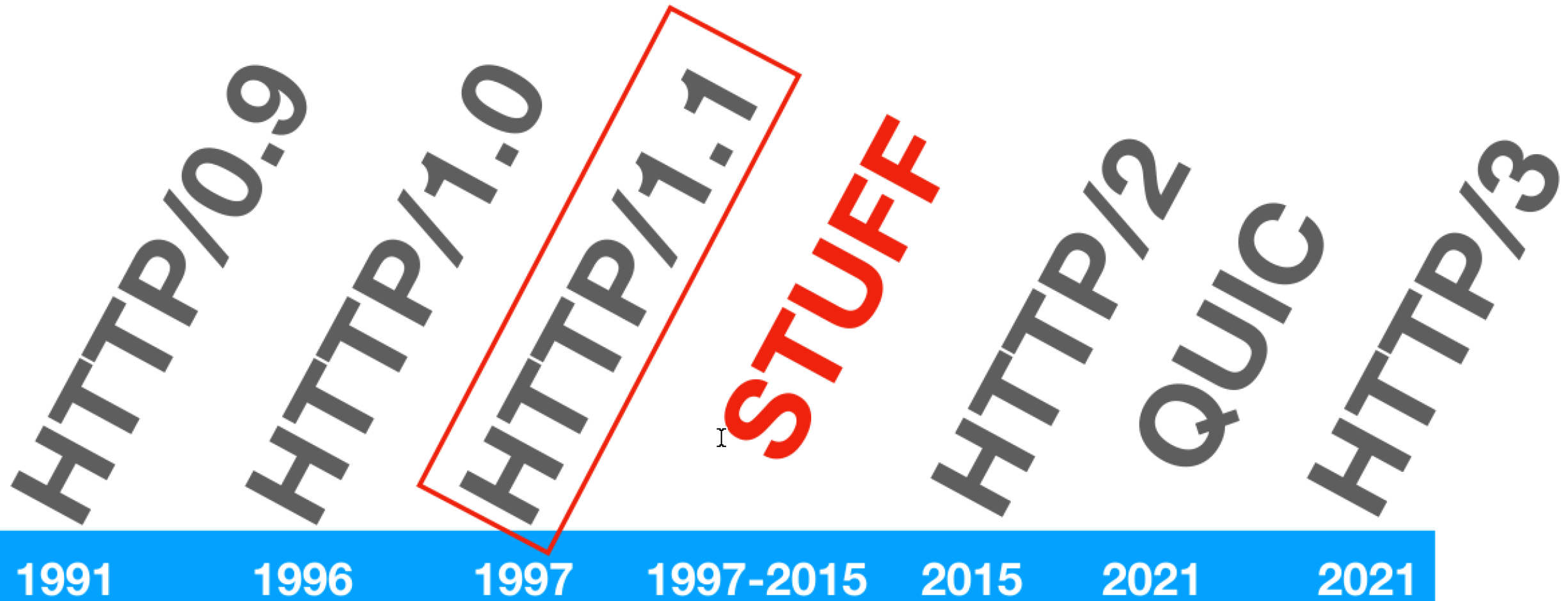
Problems!!!

- HTTP is stateless
 - Each request requires separate TCP connection
 - Server doesn't remember previous requests
- Connections were closed after requesting a single resource, this made things frustrating, slow, and expensive as websites started hosting dozens of files
- Internet connection speeds were slow, and TCP slow start had just been rolled out widely
- People wanted to host multiple websites at the same IP address, which wasn't possible (e.g., colocation services)

Which is not a bad idea if you want to scale in a hugely distributed system, we will discuss that later on, but for now let's assume it is a *problem*



Protocol Evolution



HTTP/1.1 – (1997)

Protocolo

Verbo, human readable, e foi evoluindo.

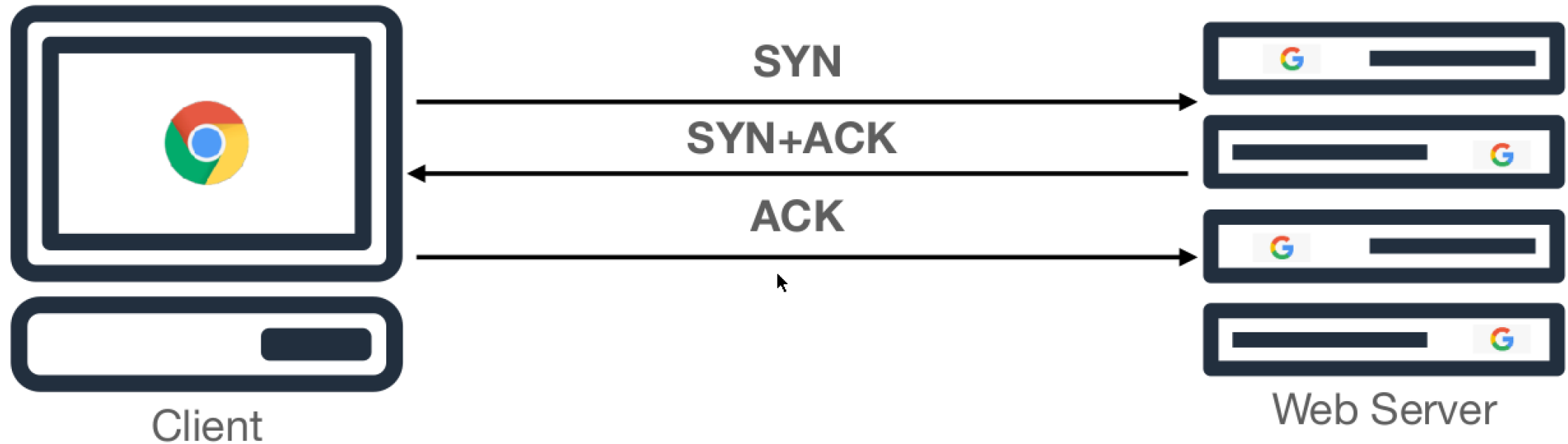


CRUDE/POST

- HTTP/1.1 fixed many problems and challenges with early versions of the protocol
 - Added the Host header (to enable multiple websites with different domains to be served from the same IP address)
- Allowed for persistent connections
- Allowed chunked responses
- Enabled pipelining of requests



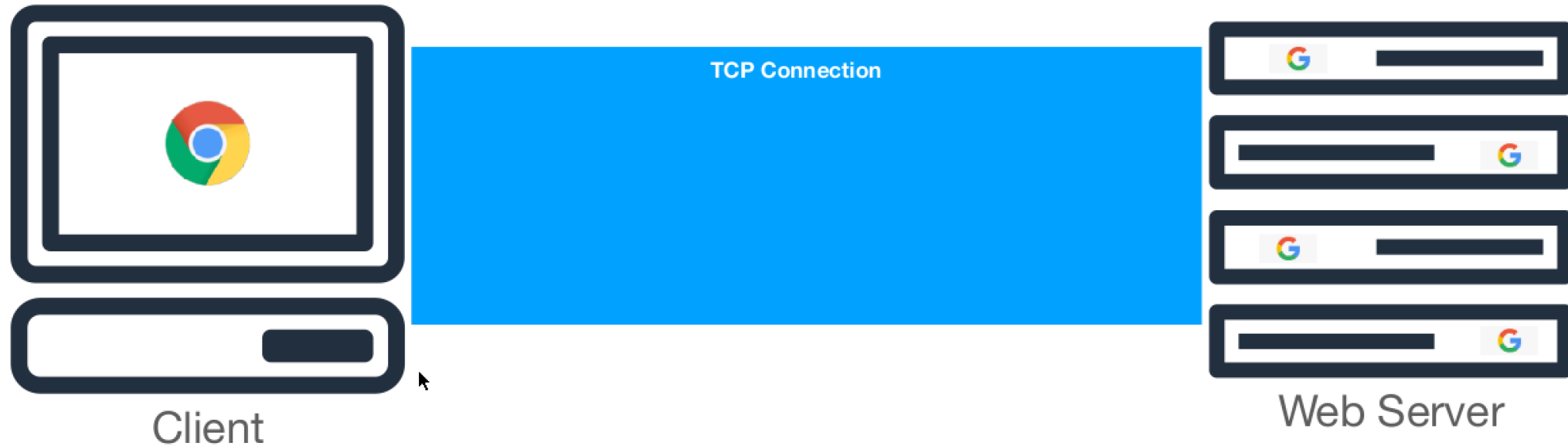
HTTP/1.1 – Persistent connections!



What is represented in this pic? What is the protocol?



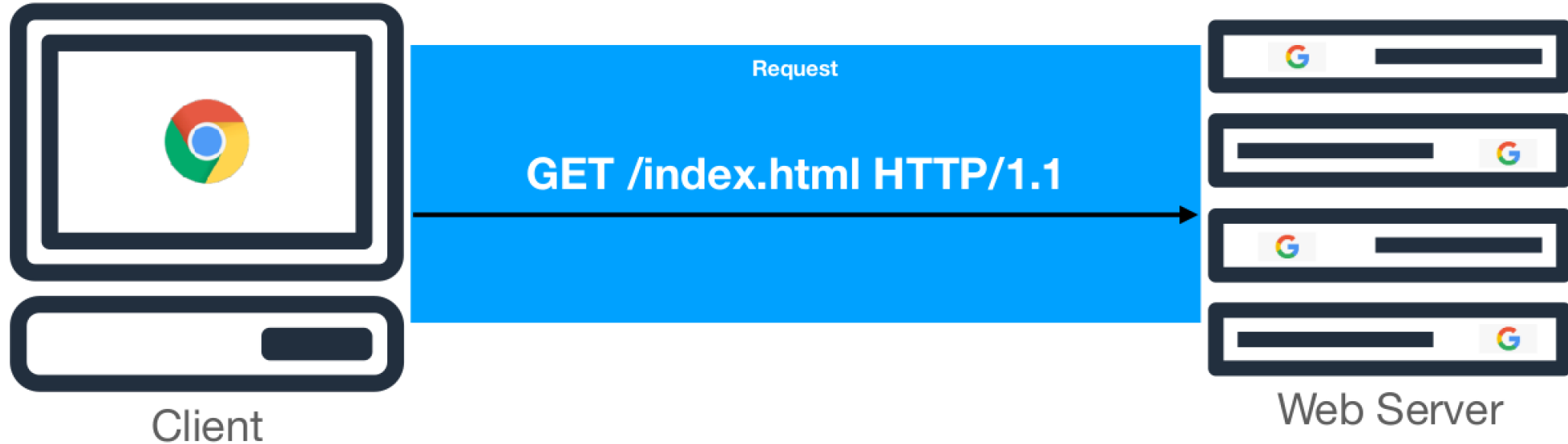
HTTP/1.1 – Persistent connections!



What is represented in this pic? What is the protocol? - is it **clear now!**



HTTP/1.1 – Persistent connections!



```
GET /index.html HTTP/1.1
Host: kumarde.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.9; rv:50.0)
Gecko/20100101 Firefox/50.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://developer.mozilla.org/testpage.html
Connection: keep-alive
```



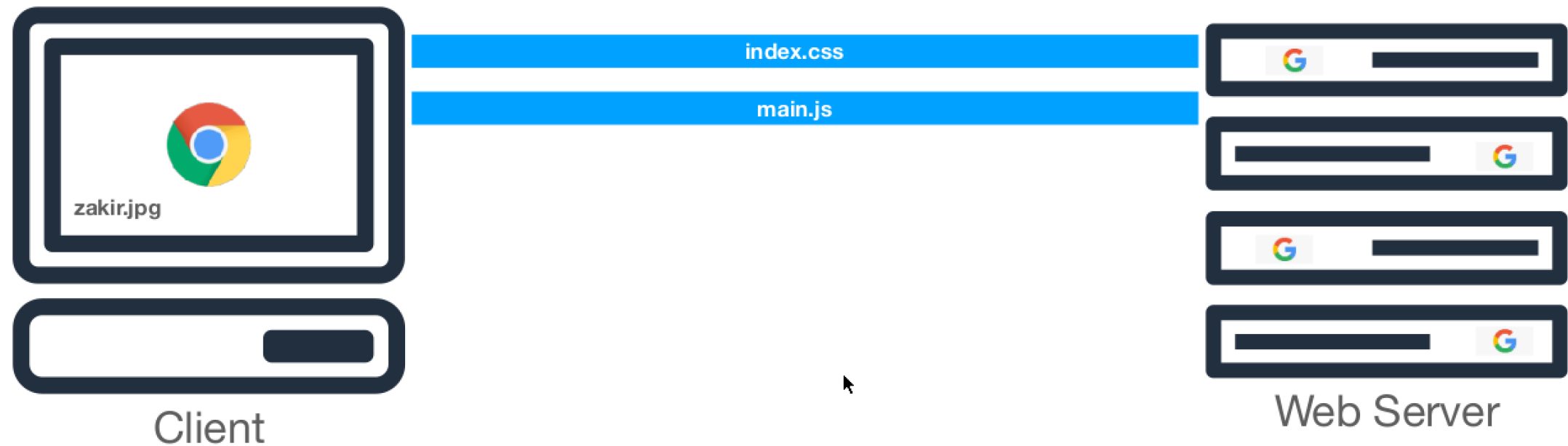
HTTP/1.1 – Persistent connections!



```
200 OK
Access-Control-Allow-Origin: *
Connection: Keep-Alive
Content-Encoding: gzip
Content-Type: text/html; charset=utf-8
Date: Mon, 18 Jul 2016 16:06:00 GMT
Keep-Alive: timeout=5, max=997
Last-Modified: Mon, 18 Jul 2016 02:36:04 GMT
Server: Apache
Transfer-Encoding: chunked
```




HTTP/1.1 – Persistent connections!



Yes, same TCP connection to Request and Response "index.css" and "main.js"
What is the advantage of it?



HTTP/1.1 – Persistent connections!

Modern browsers will open up to 6 TCP connections per host, plus 4 external TCP connections at a time





HTTP/1.1 – Chunking!

With persistent connections, servers could also now chunk data by sending a *Transfer-Encoding: Chunked* header

Essentially, this means that servers can break up their responses into independent chunks – each chunk does not need to know about the other chunks in order to send correctly (e.g., this is good for TCP)

This enabled the transfer of large files via HTTP, and also enabled streaming data (e.g., video content streaming, which is typically TCP based)



HTTP/1.1 – Pipelining!

Another feature for HTTP innovation was pipelining, essentially the ability for clients to make additional requests before the response to previous requests arrived

Requirement: Servers needed to send back responses in the order they were received

HTTP/1.1 specification dictated that servers **MUST** implement pipelining

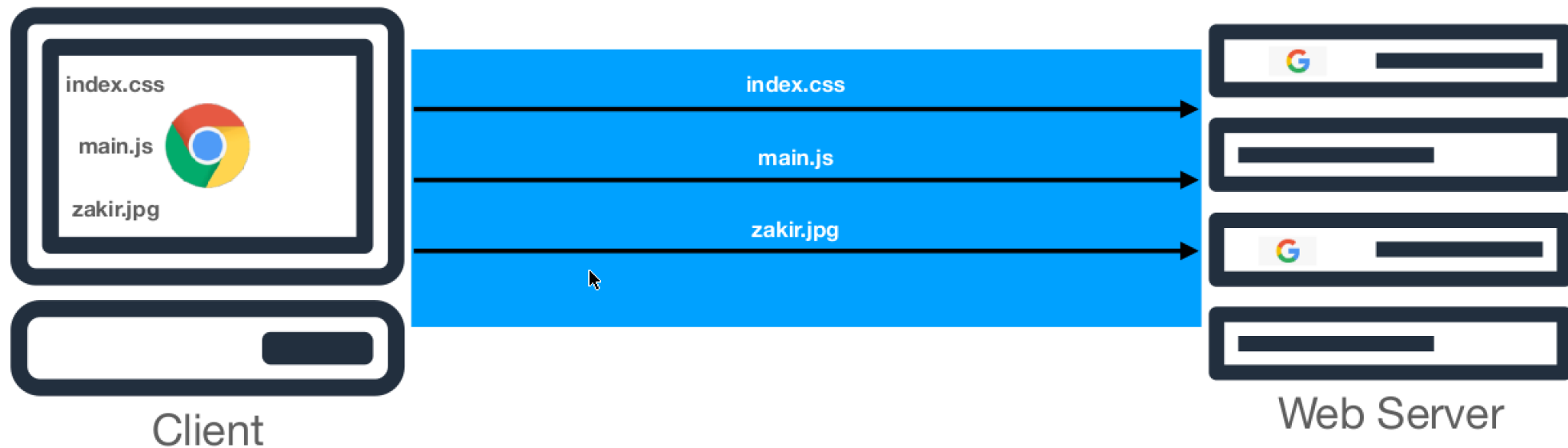
On the server side, this simply amount to keeping network buffers open and know to look for more HTTP requests on the TCP connection before response

Clients did not want to deal with HTTP pipelining... why?



HTTP/1.1 – Pipelining!

With pipelining, I can use just one TCP connection!



We are avoiding the TCP overhead (the handshake, etc)



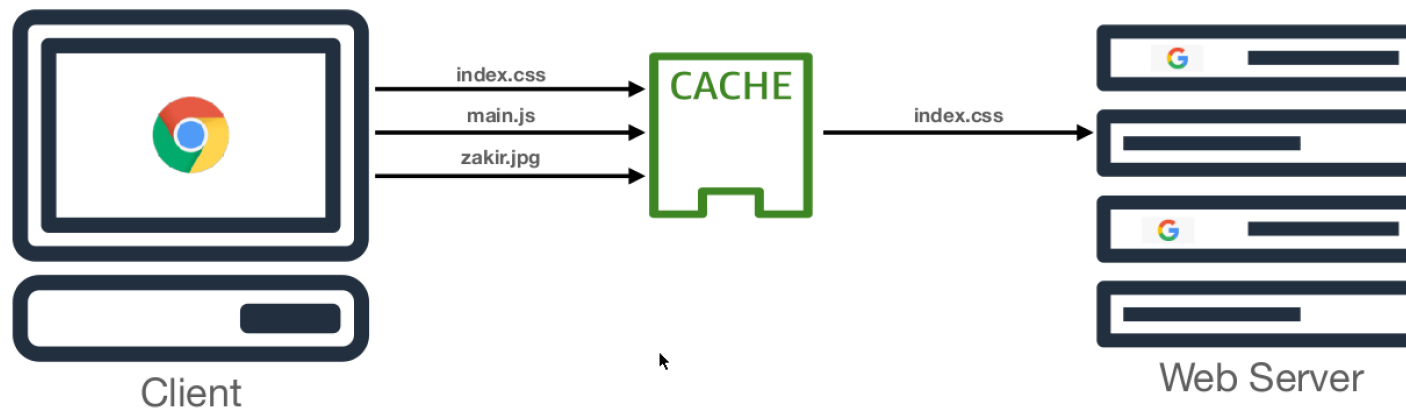
HTTP/1.1 – Pipelining!

With pipelining, I can use just one TCP connection!



But what if “index.css” takes a lot of time to retrieve? Remember that ***“Servers needed to send back responses in the order they were received”***

Also, what about HTTP proxies?





HTTP/1.1

Head of line blocking problem

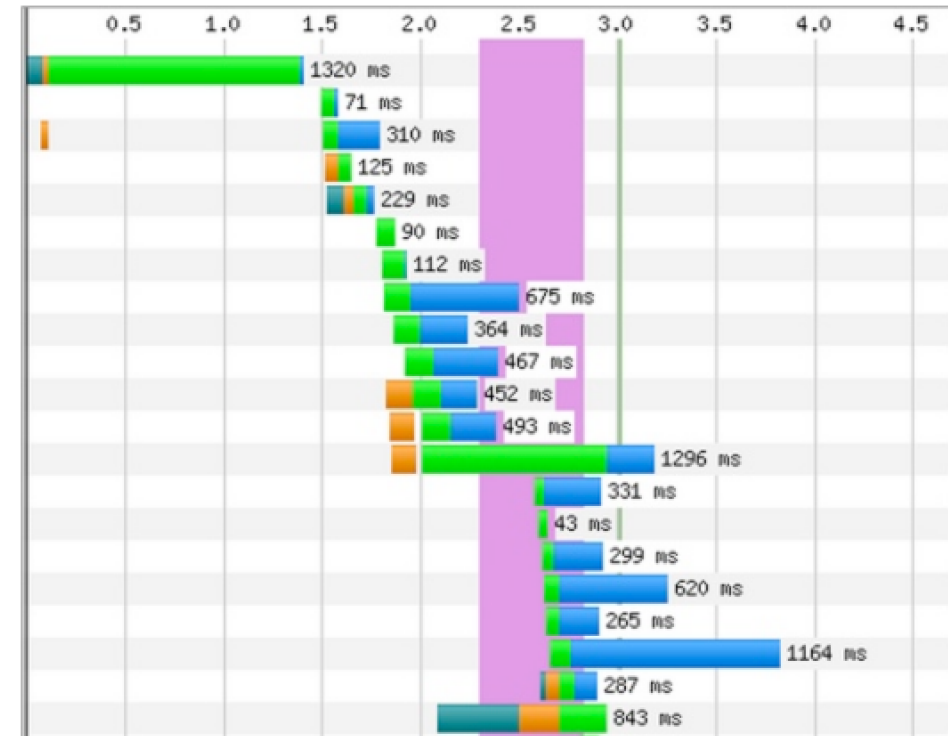
- Big problem with HTTP/1.1 pipelining is a concept called **head of line blocking** (HOL) which essentially means that subsequent resources on a shared connection need to wait for the first request to be serviced before they can be served
- In theory, pipelining is a good idea, but there are some thorny edge cases If proxies do not support pipelining, clients need to retransmit or fall-back to non-pipelining, which is hard to identify and causes delays
- This crippled HTTP/1.1 pipelining, so much so that no browsers currently support it and browser developers get angry when you bring it up



HTTP/1.1

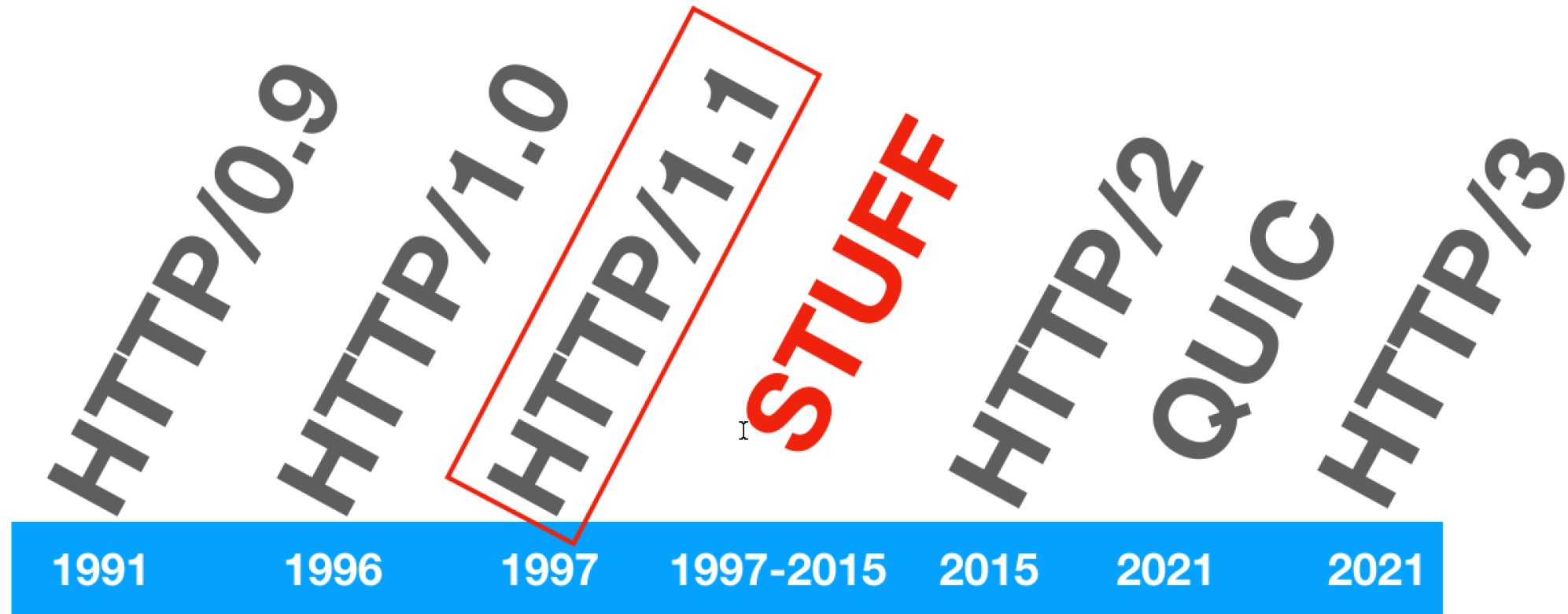
Head of Line Blocking

- Head of Line blocking is **broader** than pipelining – today, modern browsers still only open a maximum of 6 connections and have to wait for requests to finish before issuing new ones
- This is obviously slow,





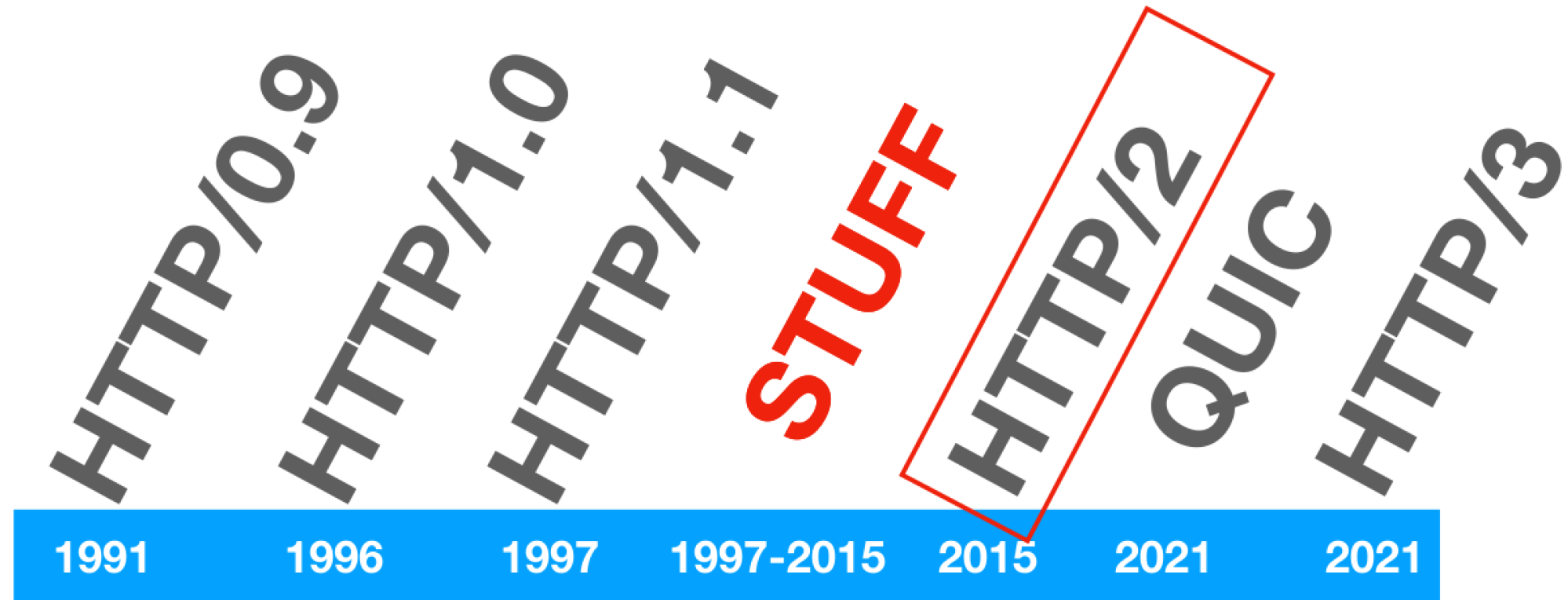
Protocol Evolution



Web evolved, more people accessing resources, let's move on...



Protocol Evolution



Now, it is getting serious!

Next slides show a 1000 feet view on HTTP/2 (go and read the RFCs)

HTTP/2

Design Goals

Pacotes



Algoritmo
Slow-start do TCP

Sempre que houver perda de pacotes começa de novo como o Slow-Start até encontrar novamente o ponto ótimo de envio de pacotes.

- Eliminate HoL blocking by multiplexing HTTP requests over a single TCP connection
- Give servers more agency (e.g., allow them to push content over persistent connections)
- Reduce unnecessary duplicate bytes sent over the wire (e.g., static headers)





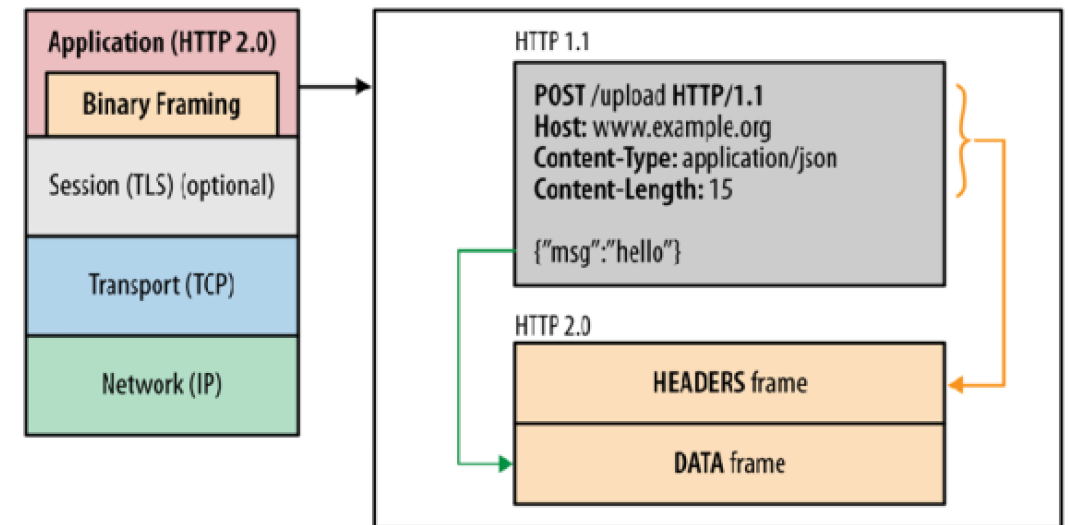
HTTP/2

Multiplexing Requests

- Move away from an ASCII-based request / response cycle for data transfer, and move towards a binary stream of data
 - ofc, not backwards compatible with HTTP/1.x

New terminology

1. Streams: A bidirectional flow of bytes which can carry one or more messages, denoted by an integer `stream_id`
2. Message: Complete sequence of frames that map to a logical request or response
3. Frame: Smallest unit of data, can contain either header information or content information

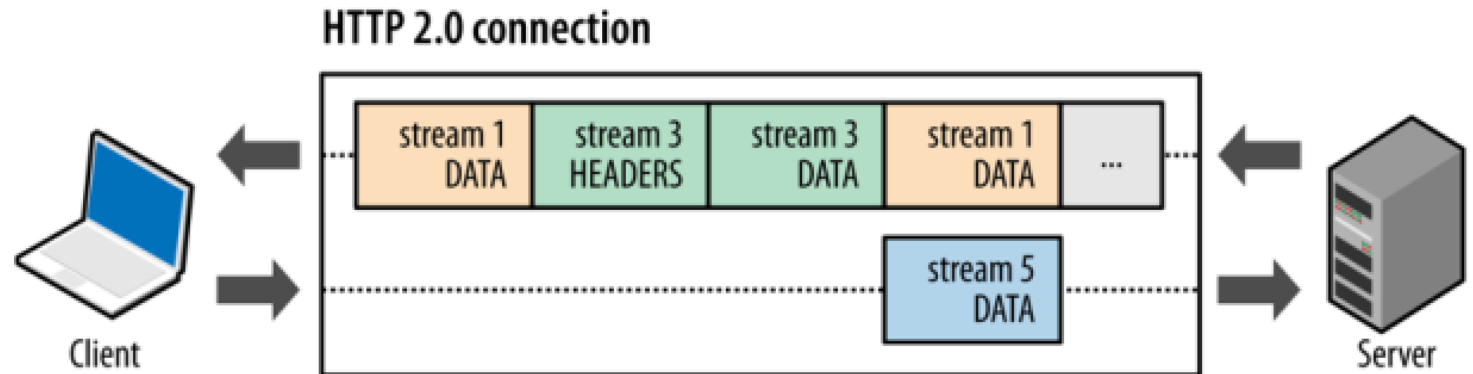




HTTP/2

Multiplexing Requests

- HTTP/2 uses a single TCP connection for any number of arbitrary HTTP requests and responses
- Everything is logically separated by stream_id (4 byte integer); This means that if the server takes significant amounts of time for one request (say, the first one), other requests can still be completed while we wait for that one!





HTTP/2

Design Goals

- ~~Eliminate HoL blocking by multiplexing HTTP requests over a single TCP connection~~
- Give servers more agency (e.g., allow them to push content over persistent connections) — did you hear about WebSockets?
- Reduce unnecessary duplicate bytes sent over the wire (e.g., static headers)



HTTP/2

Giving servers more agency

- HTTP/2 offers a new feature called Server Push, which enables the server to send data to the client that it hasn't even requested yet.

Why might we want this?

- Despite the fact that websites are highly dynamic, they still serve lots of static content (e.g., index.css, main.js)
- The server knows the client will need these assets to load the page, so why not just give it to them in advance?



HTTP/2

did you hear about WebSockets?

Give servers more agency (e.g., allow them to push content over persistent connections)

Funções de : polling
event-driven

Atualizações automáticas de páginas



HTTP/2

Design Goals

- ~~Eliminate HoL blocking by multiplexing HTTP requests over a single TCP connection~~
- ~~Give servers more agency (e.g., allow them to push content over persistent connections)~~ — did you hear about WebSockets?
- Reduce unnecessary duplicate bytes sent over the wire (e.g., static headers)



HTTP/2

Remove duplicate information as much as possible

- In HTTP/1.x, headers are always sent as plain text, despite the fact that many are static and unchanging
- We already compress application data (e.g., with Content-Encoding: gzip), but we don't do this for headers @ the protocol level
- HTTP/2 solves this with a new compression algorithm, HPACK, which has two main ideas:
 - Compress header data (Huffman coding)
 - Keep a shared compression table on the client + server that is dynamically updated with new requests every on every request / response

HTTP/2

HPACK Compression Table

- HPACK encodes a static table with 61 entries for the most common HTTP headers (and some other freebies, like GET, POST) into every client and server
- You no longer have to send these headers in cleartext, you can just send the encoded value of the index instead
- After this, every subsequent request is dynamically encoded and added to the shared table, which reduces the amount of data required to be sent over the wire for subsequent requests

Em vez de se usar letras para as operações, passam a usar-se números



Index	Header Name	Header Value
1	:authority	
2	:method	GET
3	:method	POST
...		
28	content-length	
38	host	
61	www-authenticate	
62	Host	<u>kumarde.com</u>



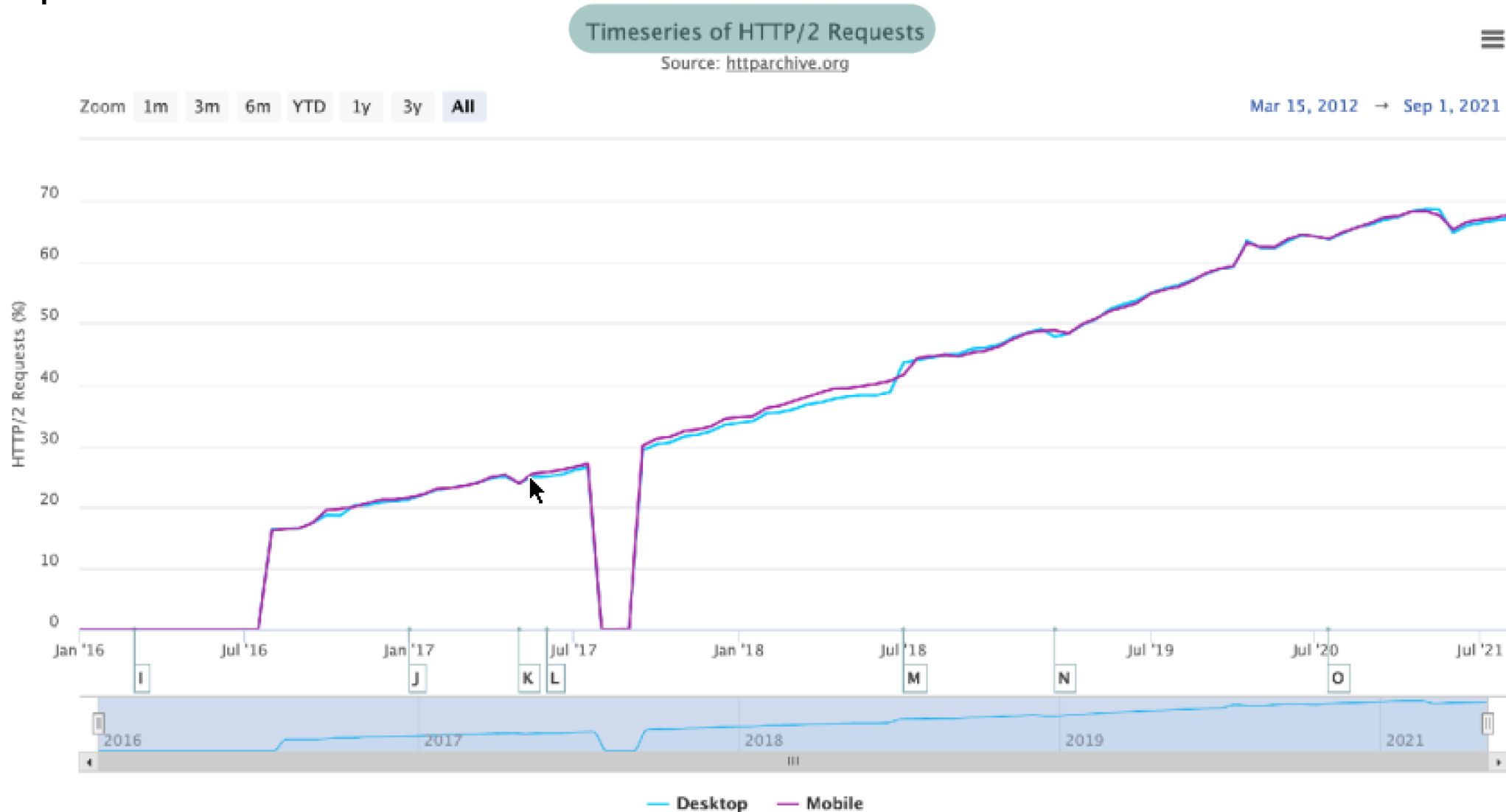
HTTP/2

Design Goals

- ~~Eliminate HoL blocking by multiplexing HTTP requests over a single TCP connection~~
- ~~Give servers more agency (e.g., allow them to push content over persistent connections)~~ — did you hear about WebSockets?
- ~~Reduce unnecessary duplicate bytes sent over the wire (e.g., static headers)~~

HTTP/2

Adoption



HTTP/2

Does it work?

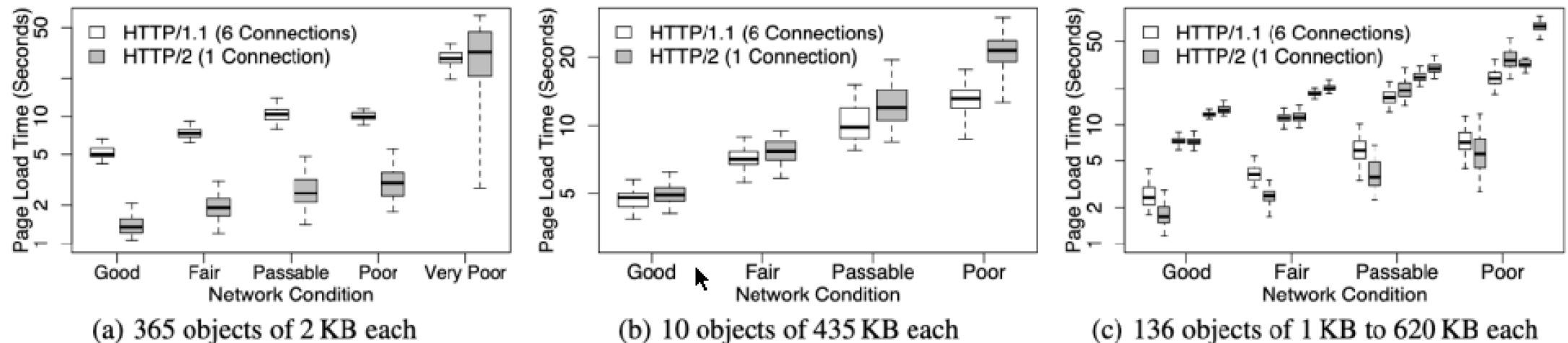


Figure 1: Distribution of page load times when loaded over h1 and h2 under various network conditions.

- “HTTP/2 Performance in Cellular Networks”, from Montana State + Akamai, showed that in poor network conditions, HTTP/2 performed **worse** than HTTP/1.1, especially for larger objects. **Why?**

HTTP/2

A new problem

- HTTP/2 solves the HTTP-level HoL blocking problems associated with older versions of HTTP... but introduces a new problem at a lower layer

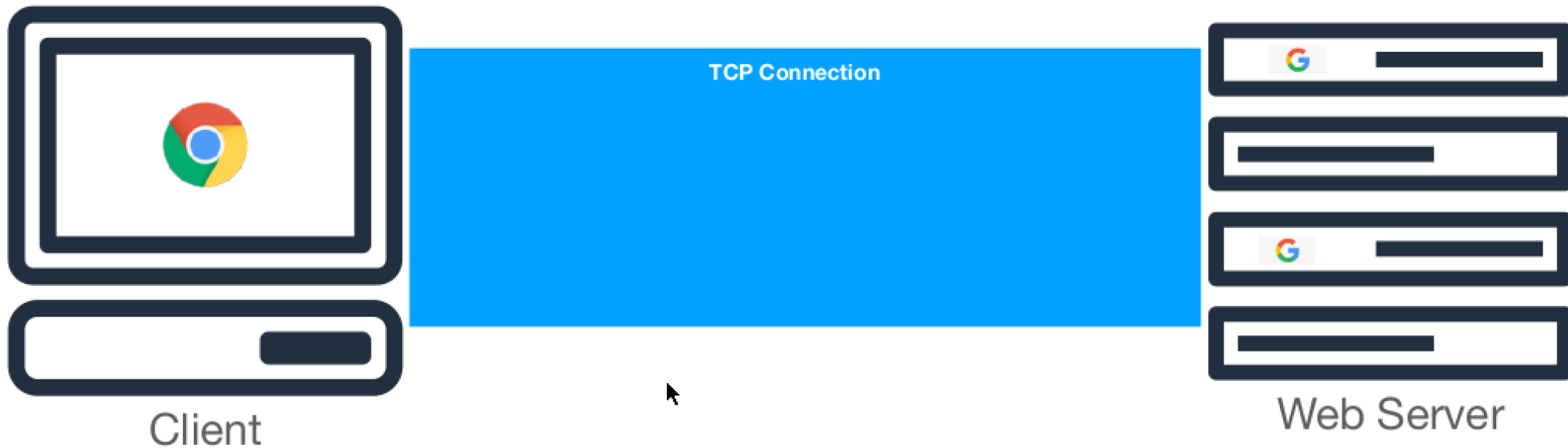




HTTP/2

A new problem

- HTTP/2 solves the HTTP-level HoL blocking problems associated with older versions of HTTP... but introduces a new problem at a **lower layer**

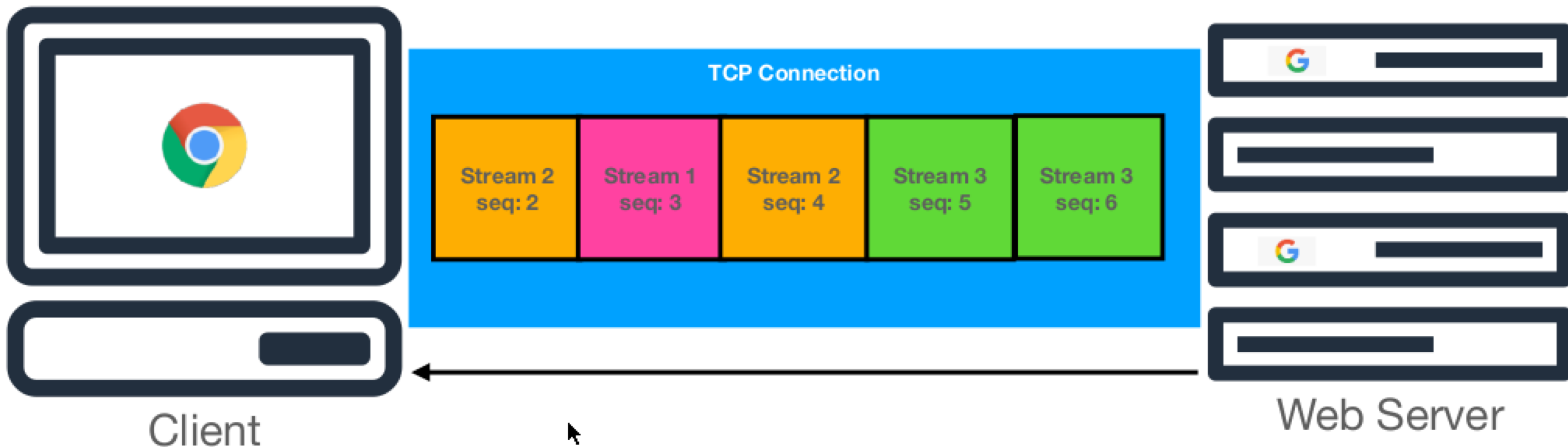




HTTP/2

A new problem

- Streams in the same TCP Connection – why is this a problem?

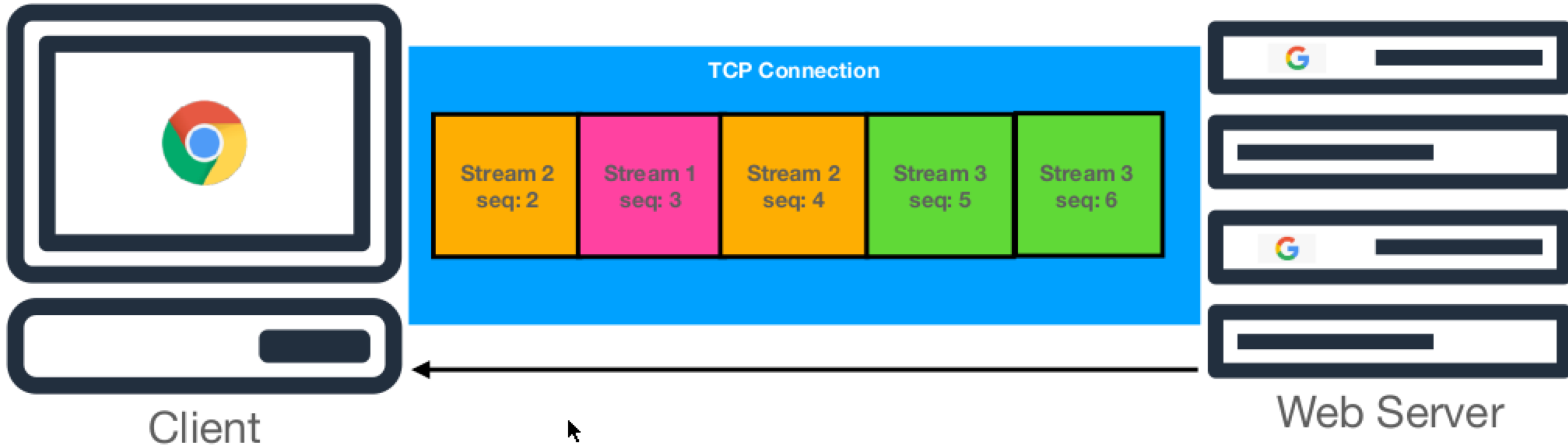




HTTP/2

A new problem

- Streams in the same TCP Connection – why is this a problem?

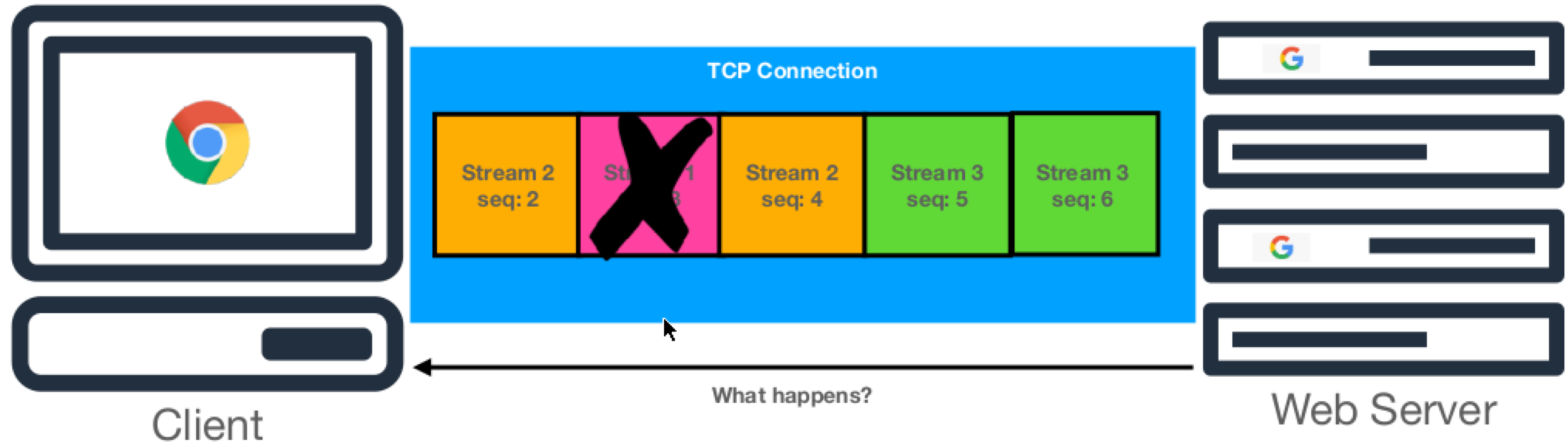




HTTP/2

A new problem

- What happens to the TCP connection if a Stream is lost? What is a Stream?

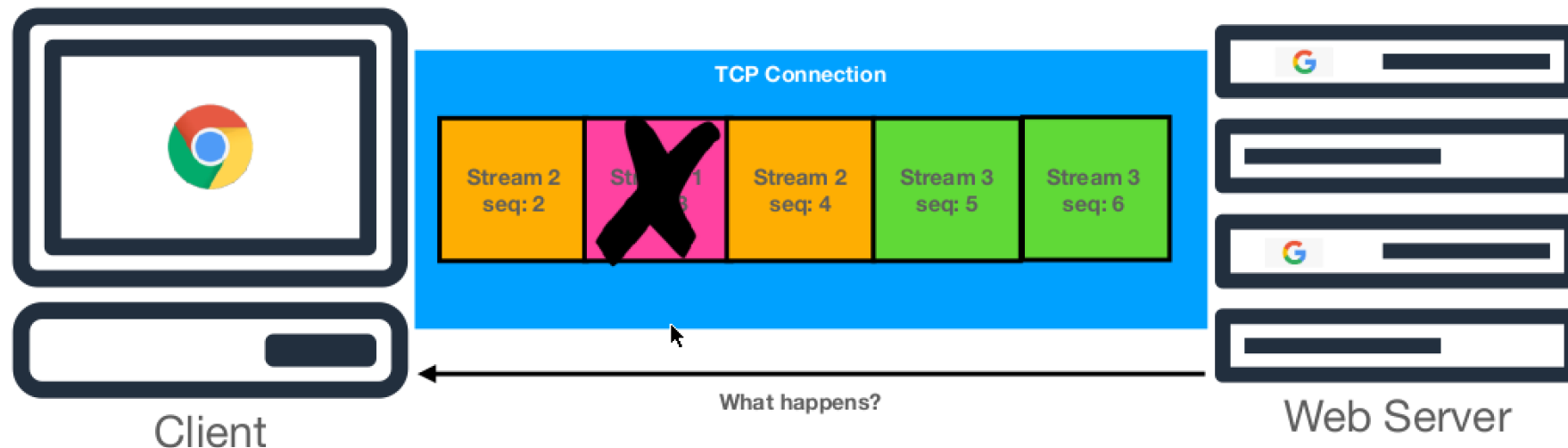




HTTP/2

A new problem

- What happens to the TCP connection if a Stream is lost? What is a Stream?
- All the streams in the TCP connection need to be dropped and a new TCP connection created. In the end we don't lose 1 Stream but several Streams
- And with the new TCP connection we have a **new retransmission of all the streams**



HTTP/2

A new problem (does it work?)

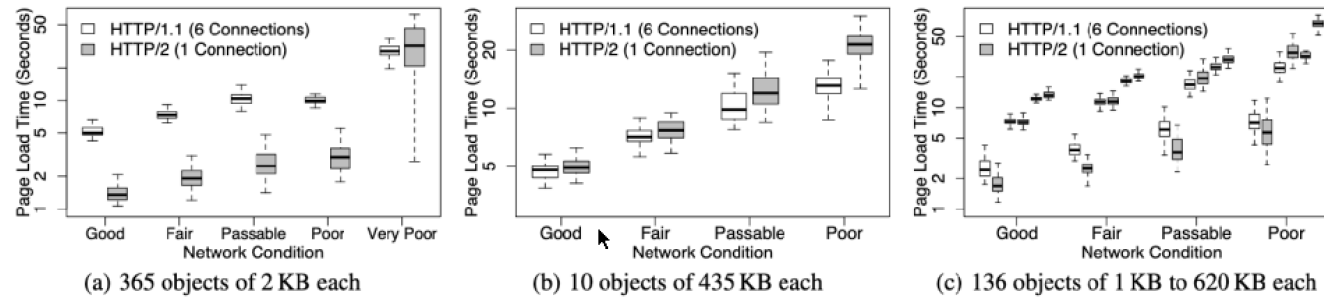


Figure 1: Distribution of page load times when loaded over h1 and h2 under various network conditions.

- “HTTP/2 Performance in Cellular Networks”, from Montana State + Akamai, showed that in poor network conditions, HTTP/2 performed **worse** than HTTP/1.1, especially for larger objects. **Why?**

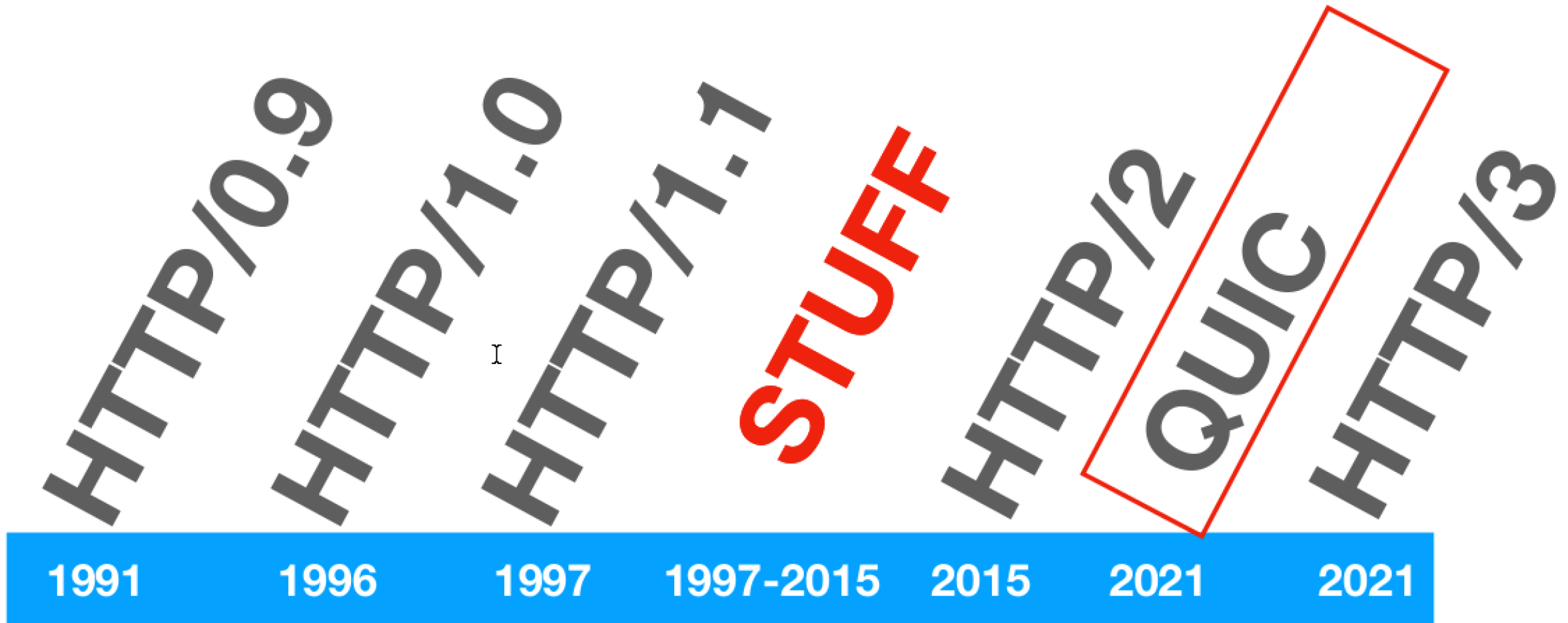
wireless tem mais
ruído/interferências
do que em redes
por cabo



And it is because of that that in Cellular Networks the problems is more visible. More drops (packet loses or unsorted delivery of packets....



Protocol Evolution





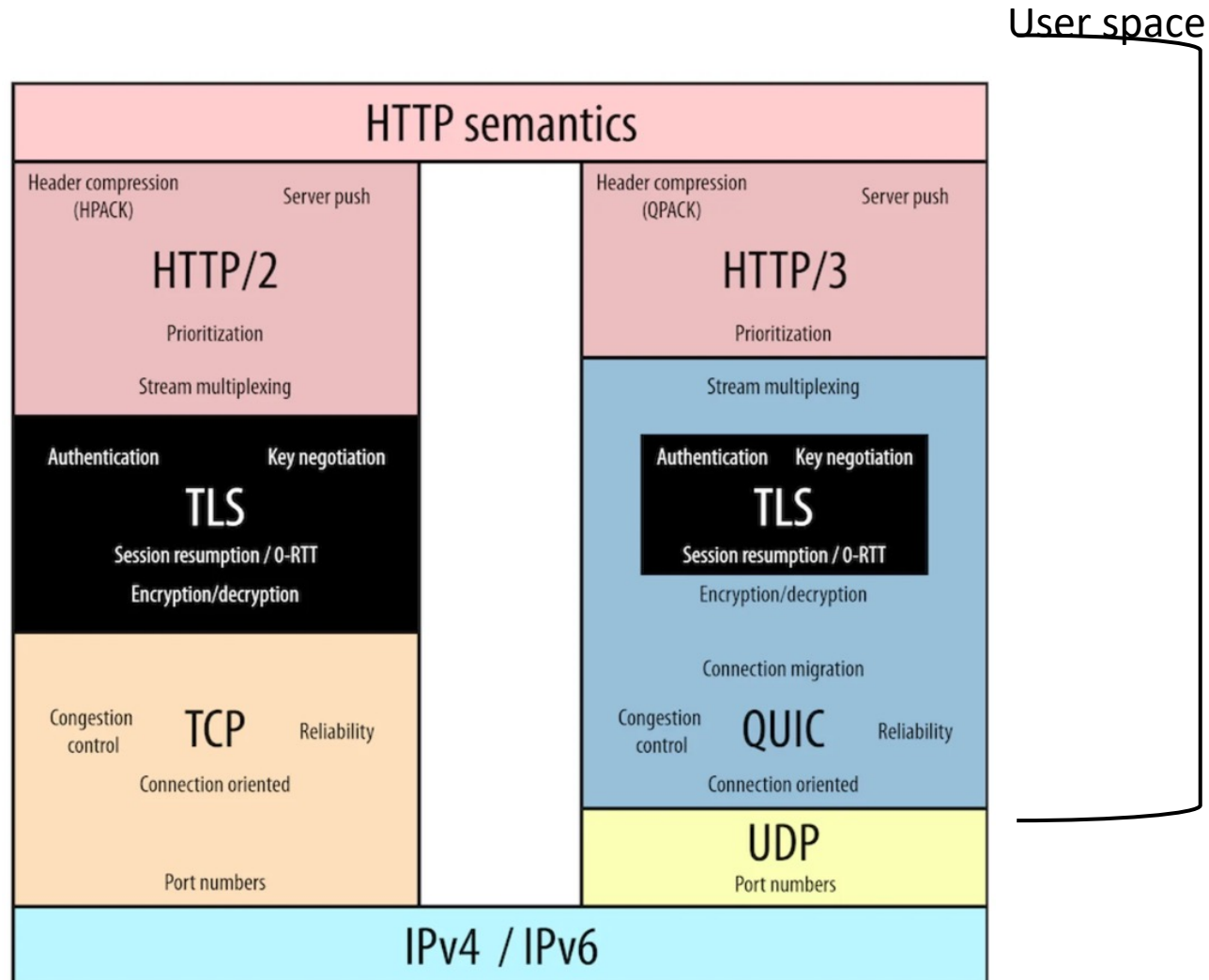
QUIC

Not an acronym, a real name

- A core problem with HTTP up to this point is a fundamental limitation of reliable transport
 - We want to have reliability guarantees, but the way this is implemented in the layering model (e.g., in TCP) makes it such that applications don't have flexibility to define what reliability means!
- We could try to change TCP?
 - But that requires updating every router in the world. Way too hard.
- **QUIC idea** - What if we re-envisioned what we needed from lower network layers?

QUIC

A new transport layer





QUIC

Design Goals

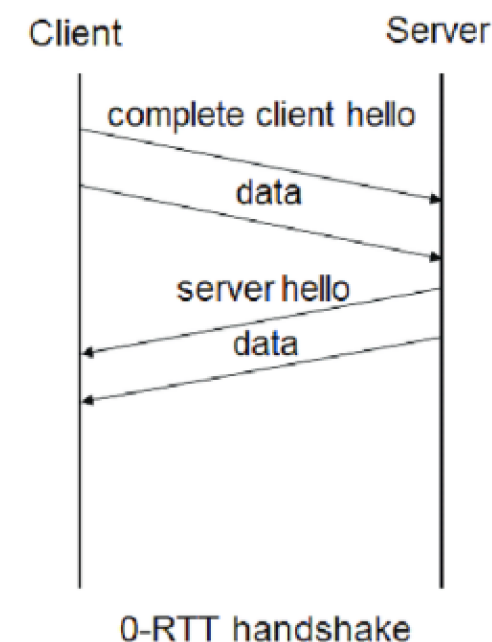
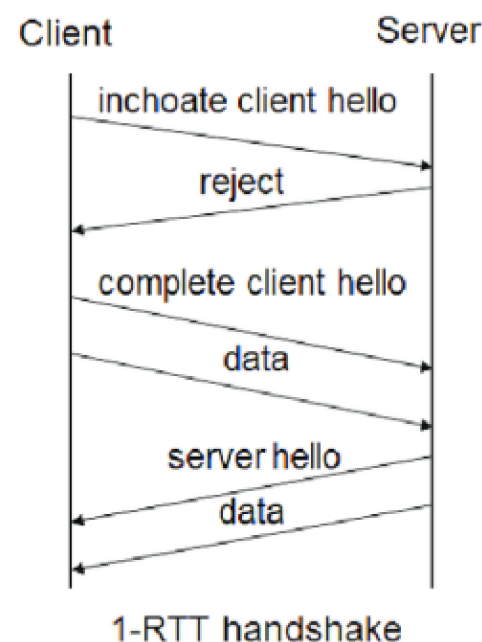
- A **new, reliable transport layer**
 - Easily deployable and evolvable
 - Make this something that exists in userspace and something that doesn't require us to update every router ever
- Security by default
 - Build in encryption, integrity checks, and authentication into the transport layer itself
- **Reduce unnecessary delays imposed by strict layering**
 - Handshake delays (e.g., TLS handshake), HoL blocking (HTTP, TCP)

QUIC

Establishing a connection



- The first time a client wants to communicate with a server, it send an *inchoate client hello* in cleartext, which will initiate a REJ (reject) from the server
 - The server will send back a number of details, including a certificate chain (for server authentication) and other server metadata
- The client will then use the server information provided to send a *complete client hello*, and immediately start sending encrypted data
- Client *caches* server details (based on origin), so for any future connection, the client can simply use the server data to send encrypted messages moving forward. This is known as a **0-RTT protocol**.





QUIC

Encrypt as much as possible

HTTP w/ TLS + TCP

source port		destination port	
sequence number			
acknowledgement number			
hlen	flags	window	
checksum		urgent pointer	
[options]			
type	version		length
length			
application data (HTTP headers and payload)			

HTTP w/ QUIC

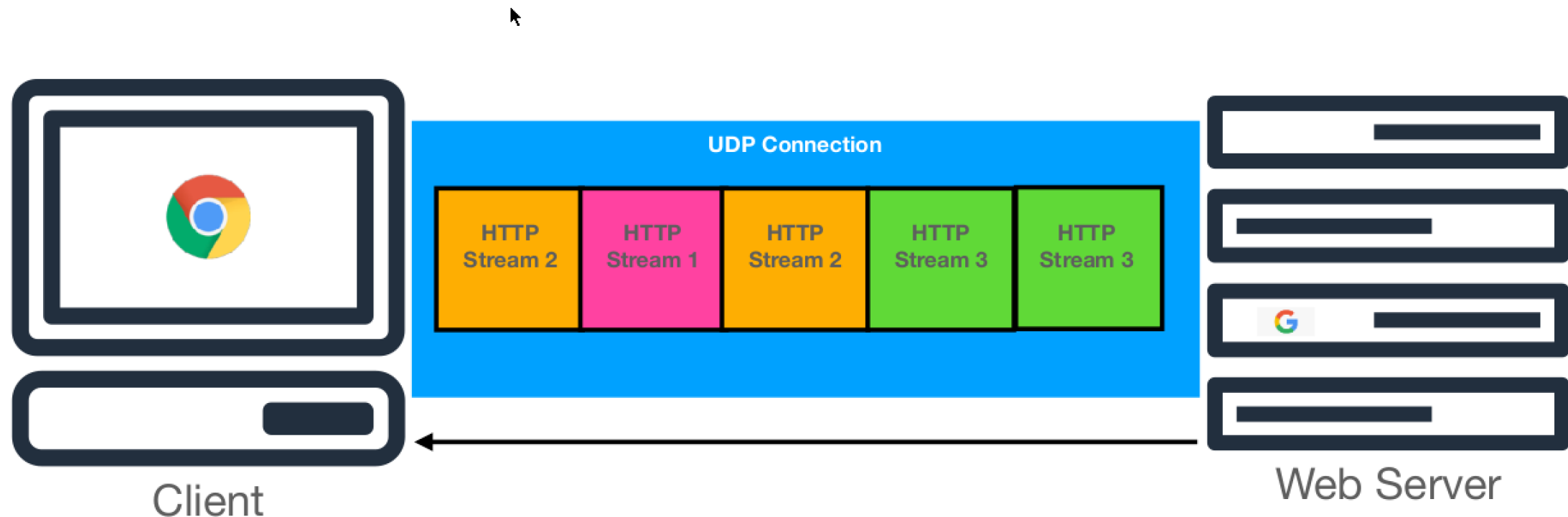
source port		destination port	
length		checksum	
01SRRKPP	[dest connection id]		
packet number			
application data (HTTP headers and payload)			



QUIC

Maintaining the Stream Abstraction

- QUIC uses the idea of a stream (with a stream_id) as a baseline abstraction for sending data between two endpoints, similar to HTTP/2

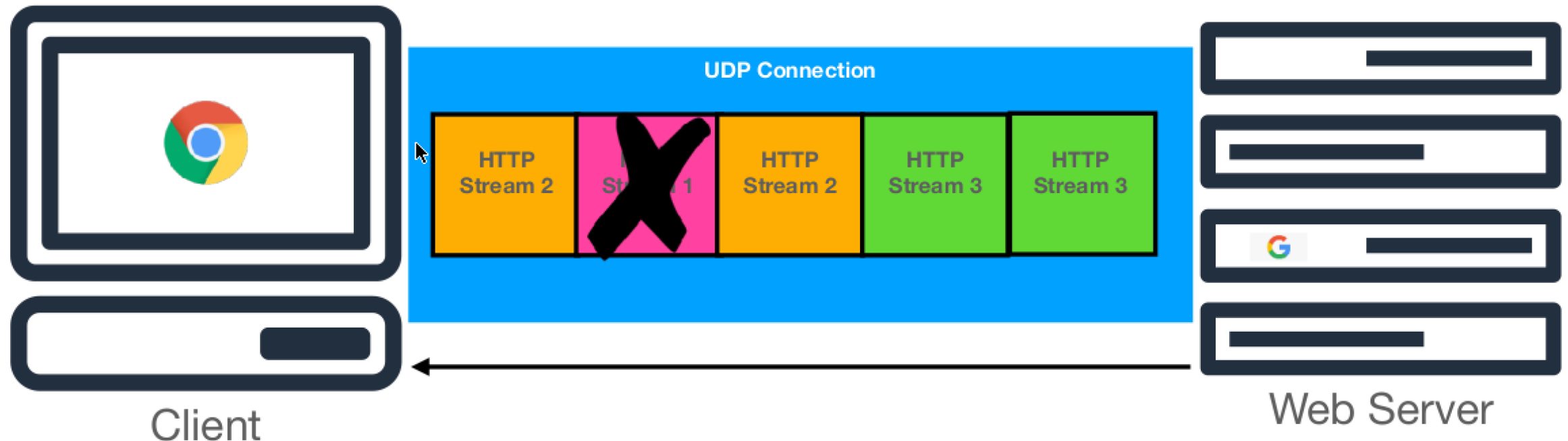




QUIC

Maintaining the Stream Abstraction

- QUIC uses the idea of a stream (with a stream_id) as a baseline abstraction for sending data between two endpoints, similar to HTTP/2

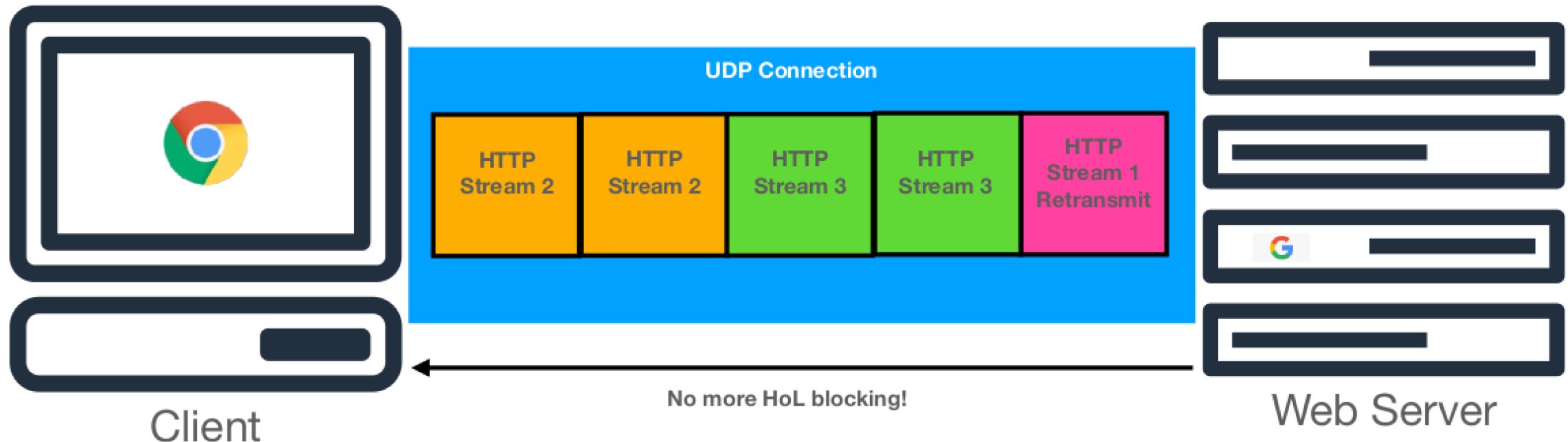




QUIC

Maintaining the Stream Abstraction

- QUIC uses the idea of a stream (with a stream_id) as a baseline abstraction for sending data between two endpoints, similar to HTTP/2





TCP vs. QUIC

Recovering from Losses

- TCP uses sequence numbers + acknowledgement numbers to identify whether or not a packet has been lost, and needs to be retransmitted
 - Unfortunately, sequence numbers mean two things: reliability **and** the order at which the bytes are supposed to be delivered to the receiver
 - On top of this, TCP retransmissions use the *same* sequence number, so it becomes very hard to know whether an ACK was sent for first transmission or a retransmission
 - TCP conflates transmission ordering AND delivery ordering in one number



TCP vs. QUIC

Recovering from Losses

- QUIC decouples transmission and delivery ordering through its use of *streams*
 - Each packet contains a packet number, which is **unique and monotonically increasing, even on retransmission**
 - Clients will ACKNOWLEDGE packet numbers, and the server can identify if an outstanding packet has not been acknowledged...
 - Each frame in a stream contains a *stream offset*, which alerts the client of how to properly reorder the packets on the delivery side
- Enables simpler loss detection than TCP



QUIC

Connection Rebinding

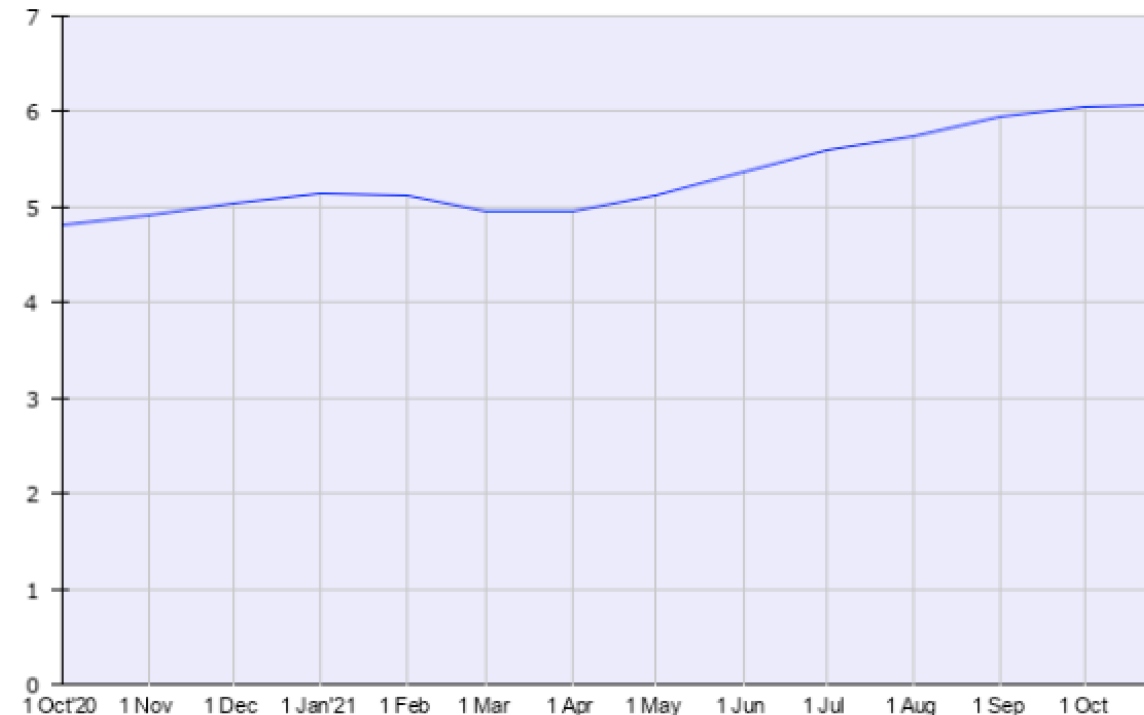
- Because QUIC connections are over UDP, they can persist *beyond traditional network boundaries*, like your home NAT
 - No more resetting connection when your underlying network changes
- QUIC does this through the use of several unique variable length Connection IDs to identify the connection, with a protocol in place to verify the connection through a network change



QUIC Deployment

QUICly eating the world

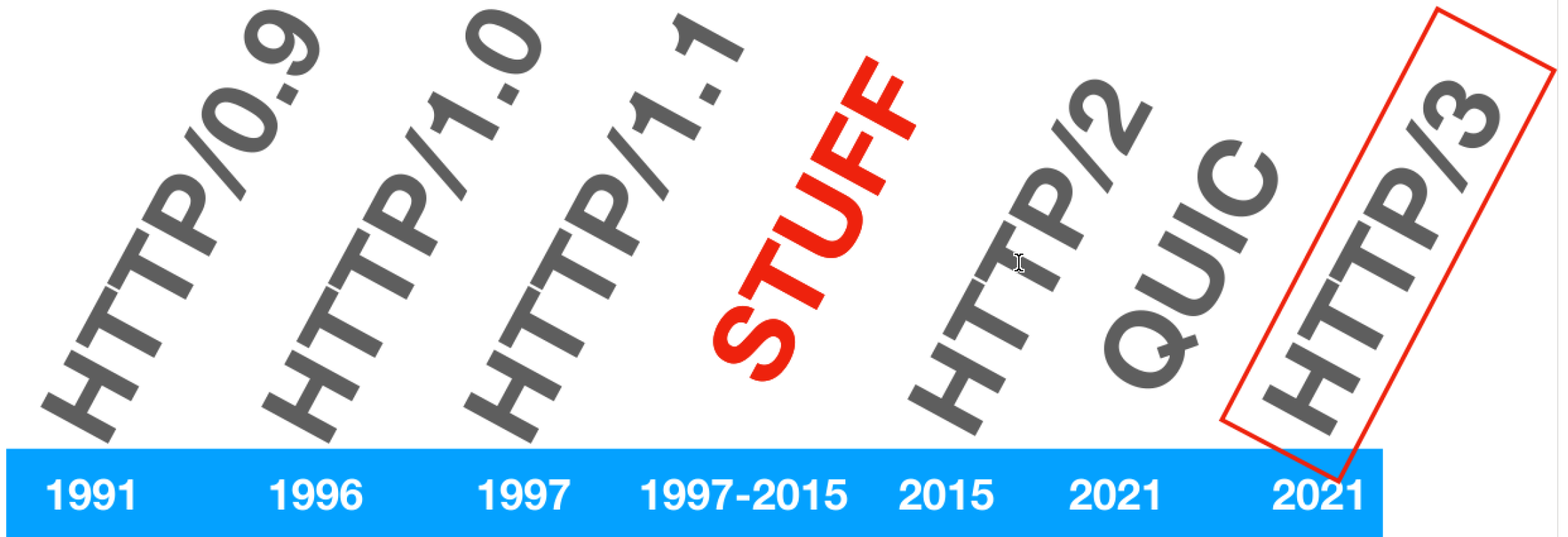
- QUIC was officially ratified by the IETF in May 2021 (RFC 9000)
- QUIC support already existed in Chrome for a while, but is now available in Firefox as well
- QUIC is being deployed everywhere
 - 6% of websites use QUIC, but will grow post RFC ratification
 - Google apps all use QUIC, 75% of Facebook uses QUIC
 - Some ISPs have reported that **20% of their packets were over QUIC**
- With appropriate tuning in high performance benchmarks, QUIC is so far as good as TLS 1.3 over TCP



Usage of QUIC for websites, 27 Oct 2021, W3Techs.com



Protocol Evolution





HTTP/3

Building HTTP over QUIC

- Still being iterated on by IETF (no RFC number yet)
- HTTP/3 uses the same abstraction as HTTP/2 (e.g., streams, frames, etc.), except it utilizes these streams as supported by QUIC rather than implementing on top of TCP
- This causes some notable new challenges:
 - HPACK, the clever header encryption scheme, cannot be enforced anymore without causing HoL blocking (recall that headers **MUST** appear before response data in HTTP/2)
 - HTTP/2 enjoyed stream prioritization, which is hard to implement in the transport layer on top of everything else



HTTP/3 vs. HTTP/2

Notable Changes?

- HPACK is updated to QPACK, which is designed to allow for out-of-order header data (and updating dynamic tables accordingly)
 - Essentially, adds more ability for client to control when to use a dynamic table entry – no need to wait to update an entry or read a table entry before processing a request
- Removed stream prioritization altogether!
 - Deemed too challenging to use for clients and offered little guarantees anyway, so it is being discussed independently



HTTP/3

- Browser Support

(<https://caniuse.com/http3>)

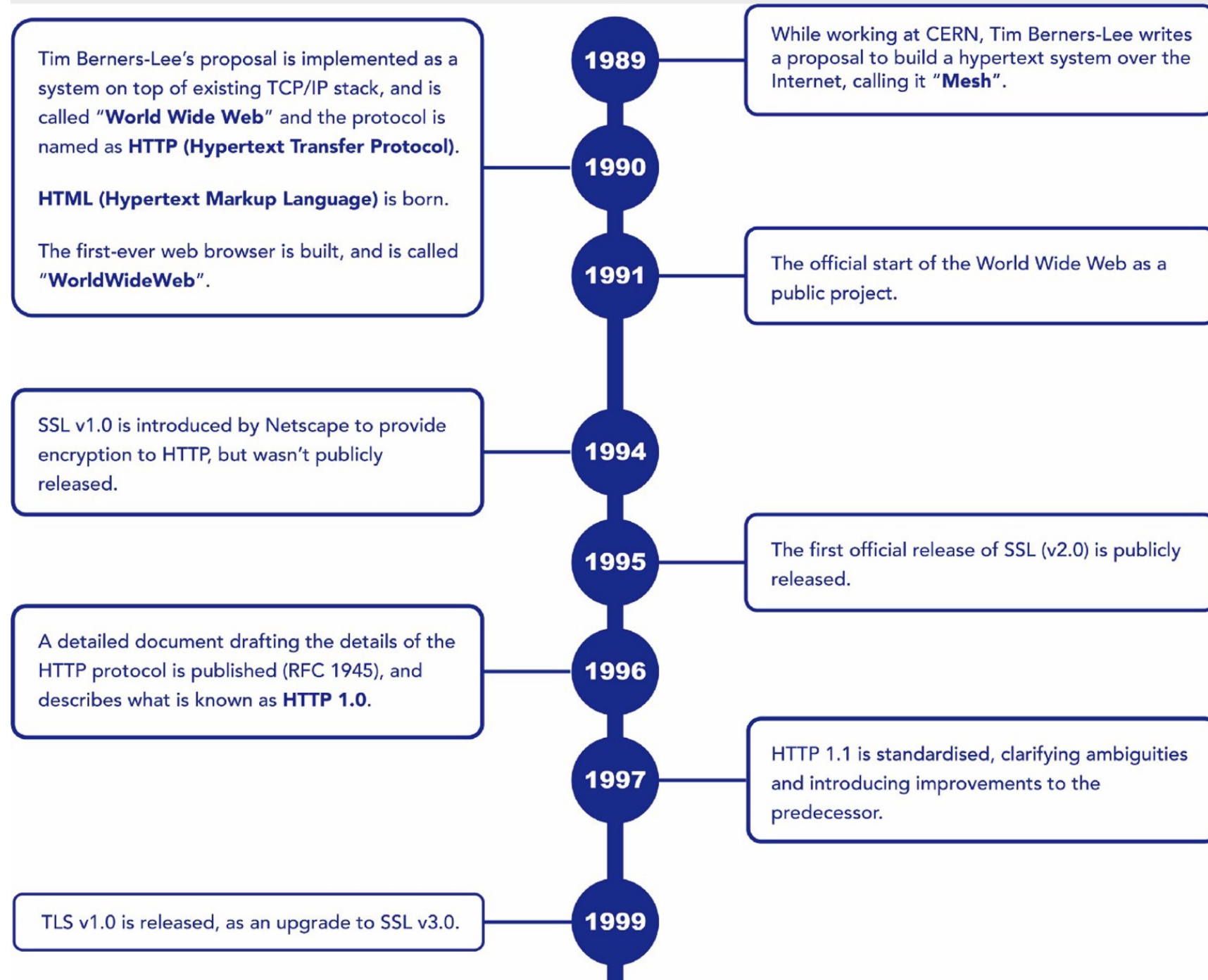
Handwritten note: *http 3*

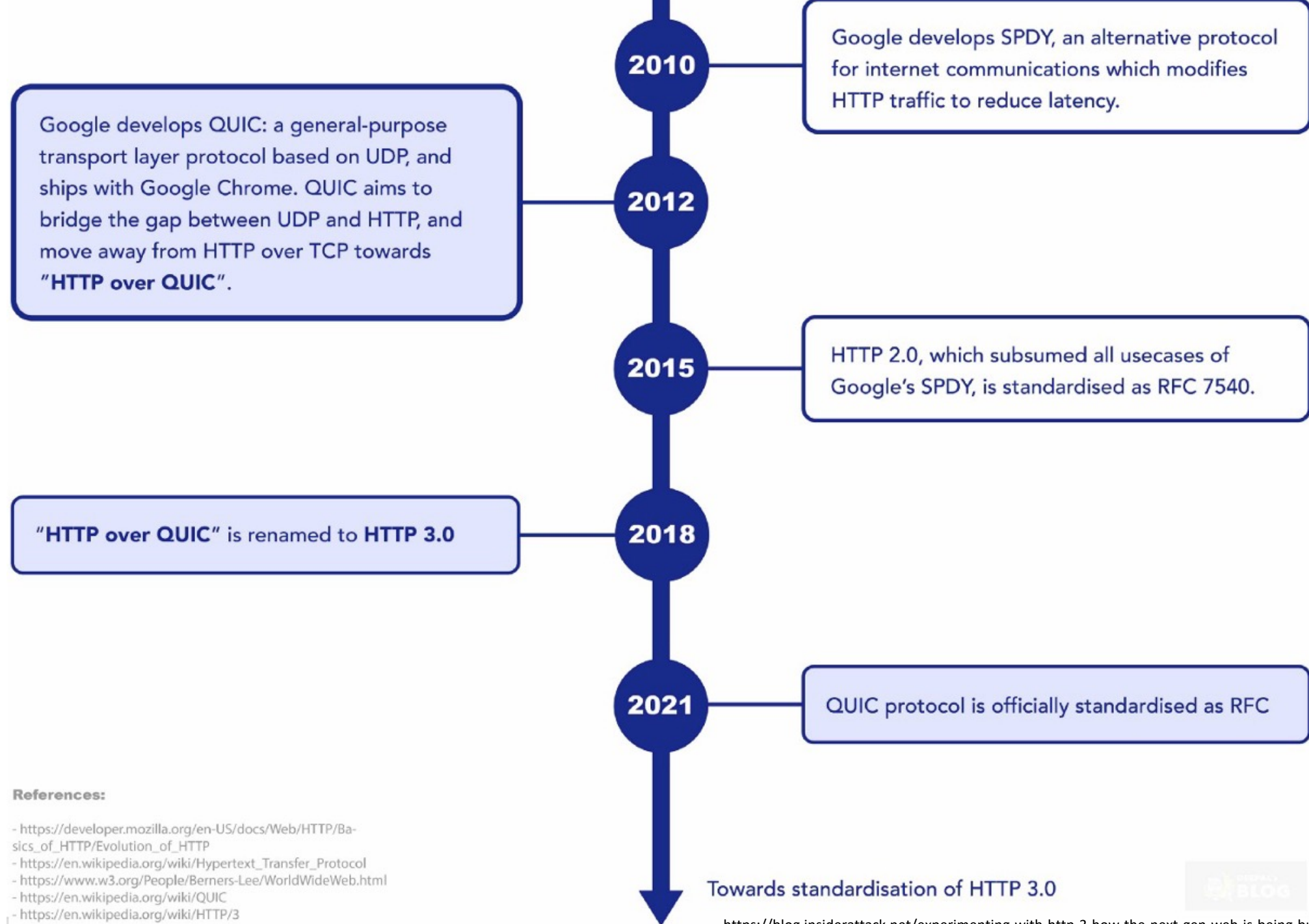
Browser Developer Tools - Headers tab

URL: www.google.com

Response Headers:

- alt-svc:** h3=":443"; ma=2592000, h3-29=":443"; ma=2592000, h3-Q050=":443"; ma=2592000, h3-Q046=":443"; ma=2592000, h3-Q043=":443"; ma=2592000
- bfcache-opt-in: unload
- cache-control: private, max-age=0
- content-encoding: br
- content-length: 53661
- content-type: text/html; charset=UTF-8
- date: Sat, 23 Oct 2021 12:17:33 GMT
- expires: -1
- p3p: CP="This is not a P3P policy! See g.co/p3phelp for more info."





Example docker & HTTP3

<https://blog.insiderattack.net/experimenting-with-http-3-how-the-next-gen-web-is-being-built-on-udp-ba01cb88a495>



Bibliography/References

Further Reading

- Slides based on material from here: <https://cs249i.stanford.edu/> (Aug2022)
- James F. Kurose, Keith W. Ross, “Computer Networking”, 6th Edition, PEARSON
- <https://www.smashingmagazine.com/2021/08/http3-core-concepts-part1/> (Aug2022)
- <https://www.smashingmagazine.com/2021/09/http3-practical-deployment-options-part3/> (Aug 2022)
- <https://http3-explained.haxx.se/en/> (Aug2022)
- <https://kevincurran.org/com320/labs/wireshark/lab-http.pdf> (Aug2022)
- <https://kevincurran.org/teaching/computer-networks/> (Aug2022)
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages> (Sept 2022)